

Microprocessor

Project

–

ARM Code Optimization for Image
Converting with Keil MDK tool

8조

박대호

유지원

최은지

김선영

목차

1. Project 개요
2. 3가지 이미지 변환 함수의 동작 방식과 block diagram
3. 3가지 이미지 변환 함수의 코드
4. ARM 코드 최적화를 위한 C코드 및 ARM 코드
5. 최적의 ARM 코드 탐색
6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석
7. 결과물들
8. 프로젝트 수행일지
9. Troubleshooting

1. Project 개요

(작성자: 박대호)

이번 프로젝트에서는 Keil MDK를 활용하여 ARM 기반 임베디드 시스템에서의 이미지 처리(Image Processing) 및 변환(Image Converting) 기능을 구현하고, 해당 기능들의 성능 최적화 및 비교 분석을 수행한다. 먼저 32비트 RGBA 이미지 데이터로부터 Red 채널의 값이 128 이상인 픽셀의 개수를 세는 알고리즘을 구현함으로써 조건 기반 이미지 분석 기능을 수행한다. 이후에는 원본 32비트 RGBA 이미지를 24비트 음화(Negative) RGB 이미지로 변환하는 알고리즘과, 동일한 원본 이미지를 16비트 그레이스케일(Grayscale) 이미지로 변환하는 알고리즘을 각각 구현하여 다양한 형태의 이미지 변환을 실습한다.

해당 알고리즘들은 C 언어와 ARM 어셈블리어로 구현되며, 각 함수의 핵심 연산은 어셈블리어를 통해 성능 향상을 고려하여 작성된다. 또한 이미지 데이터는 HEX 파일 형태로 메모리에 로딩되며, Keil MDK의 디버그 모드에서 Memory Window를 통해 데이터의 변환 결과를 실시간으로 관찰하고, 동작을 확인할 수 있도록 한다.

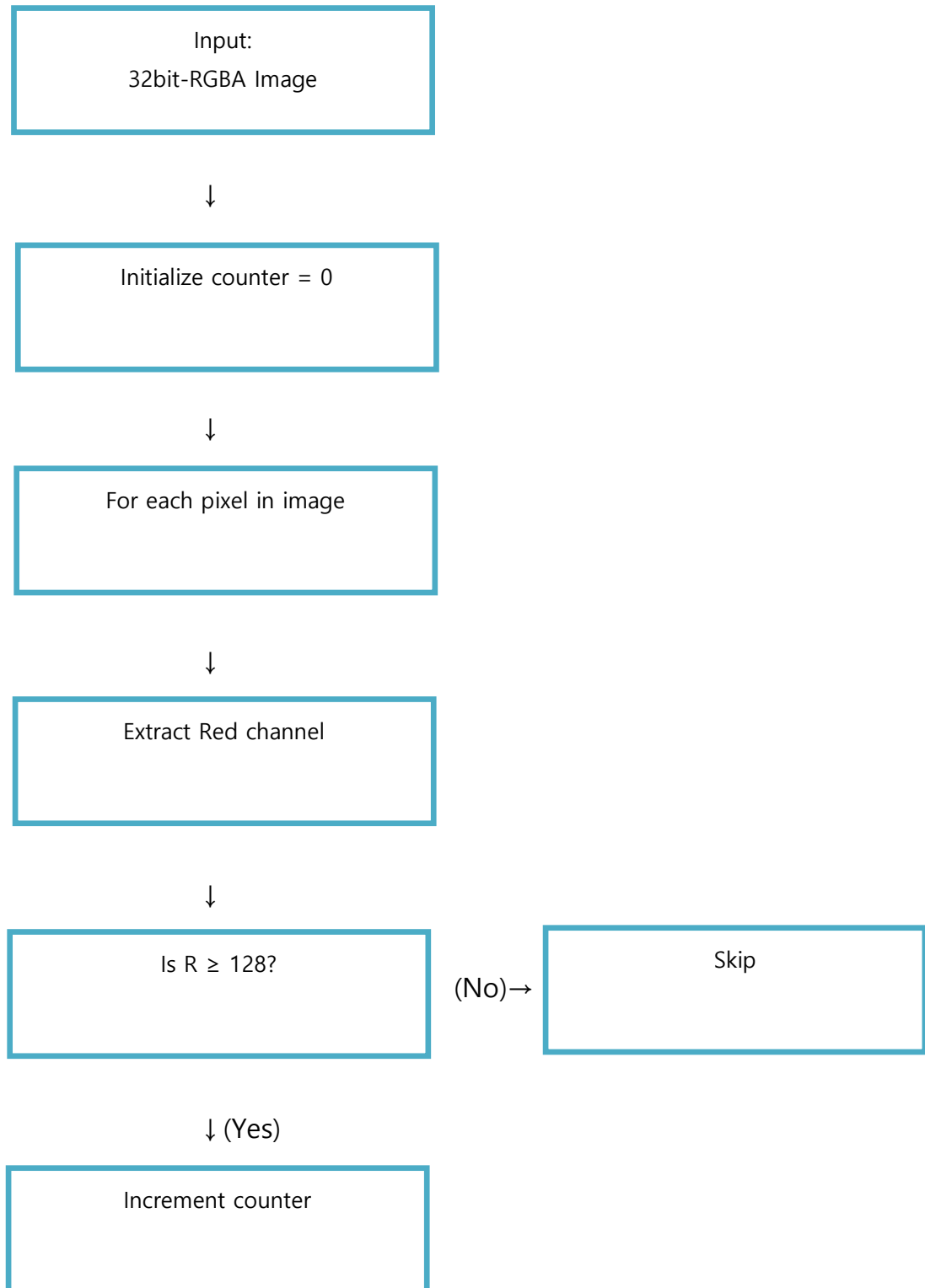
후반부에서는 작성된 이미지 처리 코드의 성능을 향상시키기 위한 최적화 기법을 적용한다. 루프 언롤링, 조건 분기 제거, 메모리 접근 방식 개선, 레지스터 활용 등 다양한 ARM 최적화 기법을 적용하며, 메모리 재배치를 통해 변환 효율을 극대화하는 방법도 실습한다. 마지막으로, 초기 버전의 ARM 코드와 최적화된 ARM 코드 간의 성능(실행 시간, 메모리 사용량 등)을 비교 분석함으로써 코드 구조가 실제 성능에 미치는 영향을 정량적으로 평가한다.

본 프로젝트를 통해 이미지 처리 알고리즘을 구현해 봄으로써, 임베디드 시스템에서의 최적화 사고력과 실전 디버깅 역량을 종합적으로 배양할 수 있을 것이다.

2. 3가지 이미지 변환 함수의 동작 방식과 block diagram

(작성자: 박대호)

(1) Subproject#1: Red Pixel Count





Repeat for 9600 pixels



Output: Total count

(2) Subproject#2: Image Negative

Input:

32bit-RGBA Image



Remove Alpha Channel

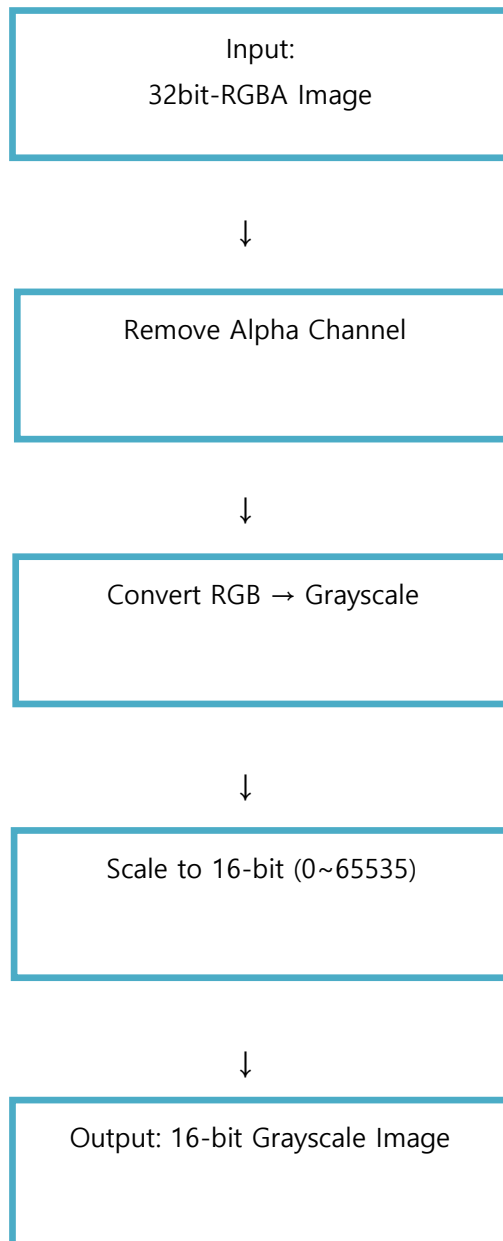


Invert R, G, B $\rightarrow 255 - R, 255 - G,$
 $255 - B$



Output: 24-bit RGB Negative Image

(3) Subproject#3: Grayscale



3. 3가지 이미지 변환 함수의 코드

Subproject 1 (작성자: 최은지)

- Assem.s

```
performance test_1.sct  assem.s
1      AREA Reset, CODE, READONLY
2      EXPORT Red_Pixel_Count
3
4      Red_Pixel_Count
5      LDR    r0, =0x40000036
6      MOV    r1, #9600    ; 총 픽셀 수
7      MOV    r2, #0      ; count
8      MOV    r3, #0      ; 현재 인덱스
9
10     loop
11         CMP    r3, r1    ; r3과 r1 비교_모든 픽셀 검사 여부 확인
12         BEQ    done     ; 같아지면 loop 종료
13
14         LDRB   r4, [r0]; r4_ 현재 픽셀의 red 값
15         CMP    r4, #128; red 값이 128 이상 인지 확인
16         BLT    skip     ; 128 미만이면 count 증가 없이 skip으로 branch
17         ADD    r2, r2, #1 ; 조건 만족 시 r2 증가_count +1
18
19     skip
20         ADD    r0, r0, #4 ; 다음 픽셀로 이동_RGBA=4B
21         ADD    r3, r3, #1 ; 인덱스 증가(다음 loop 준비)
22         B      loop     ; loop 반복
23
24     done
25         MOV    r0, r2    ; count를 r0에 복사
26         BX     lr       ; 호출한 함수로 복귀
27
28     END
```

이 코드는 RGBA 이미지 데이터에서 Red 성분이 128 이상인 픽셀 수를 세는 어셈블리 루틴이다. r0에는 RGBA 이미지 시작 주소인 0x40000036이 들어가며, 각 픽셀은 4바이트 (R, G, B, A)로 구성되어 있다. r1에는 총 픽셀 수인 9600이 저장되고, r2는 조건을 만족하는 픽셀 수를 세기 위한 카운터이며, r3은 현재 처리 중인 픽셀 인덱스이다. 루프에서는 먼저 r3과 r1을 비교하여 모든 픽셀을 검사했는지 확인하고, 아직 남아 있다면 r0가 가리키는 위치에서 Red 값을 바이트 단위로 읽어온다. 이 값이 128보다 작으면 건너뛰고, 128 이상이면 r2를 1 증가시킨다. 이후 r0은 다음 픽셀로 넘어가기 위해 4바이트 증가하며, r3도 인덱스를 위해 1 증가시킨다. 이 과정을 9600번 반복한 후, 조건을 만족한 픽셀 수가 담긴 r2 값을 r0에 복사하여 반환한다. 이 코드는 조건 검사와 주소 계산을 최소화하여 효율적으로 동작하며, 별도의 메모리 쓰기 없이 레지스터만으로 결과를 처리한다.

- 결과

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000038
CPSR	0x60000003
SPSR	0x00000000

R0 register 값 = red 값이 128 이상인 pixel의 수
: 0x12C3=4803

- 함수 수행 전 후의 memory map 정보

Memory 1	
Address: 0x40000036	
0x400000036: 78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B	
0x400000050: DF FF F3 3C B3 B5 DD 9D 6D 59 B2 02 40 AA 67 3F F7 08 80 B3 1C EB A9 2A 92 00	
0x40000006A: 32 4F 9E 13 C2 C3 A3 F9 F0 F1 B0 B5 7D 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35	
0x400000084: 86 B5 89 DD D0 C7 DC 4F D1 F6 40 B8 5D 18 CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B	
0x40000009E: 74 31 C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 E4 77 8B CB	
0x4000000B8: 14 D1 B6 31 34 6D BA BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB DB DA 55 EF D7	
0x4000000D2: 44 17 5D D3 46 34 FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F	
0x4000000EC: 6D DB 26 BA 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D 53 74	
0x400000106: 9D 5F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F	
0x400000120: 6B 22 86 46 F7 16 CD 18 FD CA 05 6C 31 F3 DE 4D E3 F5 8B 2E 5A 9B B1 3F 44 2C	
0x40000013A: 73 CC 5B C4 DC 46 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC D2 6B B7 45 D3	
0x400000154: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C	
0x40000016E: 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC 13 C3 03 6B BB B6 5A 1F	
0x400000188: 6E 5A 35 9A FB 26 FA 7B 8B 66 5C B5 E2 5B DF EA DF D0 7F 48 F4 55 EF D5 0F	
0x4000001A2: 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23 EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1	
0x4000001BC: 63 6D 87 D0 6D 3D 58 FB 86 D5 B3 D8 96 88 A1 4D 62 DB B1 3F 9C 05 61 3C C6	
0x4000001D6: AE 8B 6D 6B FA 7A 3D EB C6 F7 FA FE B9 2F EB A4 F5 72 7D 78 6F 99 F8 6D 56 FB	
0x4000001F0: 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 E1 96 78 5F BE C7 FE B8	
0x40000020A: F2 8A 6D 8B 31 06 7D 9F CB 9E F2 B3 F6 FB D7 F7 79 9E A1 E7 5E 6B CD 7D BE	
0x400000224: FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43 5D 0B DF E3 3D 16 36 42 5E D3	
0x40000023E: BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1 AD B1 B5 87 68 A2 8D 69 99 A2 E3 D8 2C 3C	
0x400000258: A3 41 F7 CB 3D F2 99 6D BB 8F 75 F1 07 D7 FA B1 F7 B5 BF B5 A6 5E 87 5F BE 62	
0x400000272: E5 01 6C BE 06 4D 88 75 8E 36 72 BF B1 6E 6C 78 2D 23 37 C4 46 D3 8E B1 C4 A4	
0x40000028C: 75 E2 87 88 1B BB 38 1C 77 B1 3F 34 F1 F5 F0 14 4F BB 0F B1 3F 9C 05 61 3C C6	
0x4000002A6: FE 61 17 8F BB 31 1E 0F FB 18 4E BB 38 EC 0F 71 1A 9E E2 B8 3F C5 E9 70 8A 71	
0x4000002C0: 38 C4 D0 8D DA BF FB F6 29 BA B1 1A 5D 3F 6B FF 75 ED 51 7B 6F D0 91 7B 89 A6	
0x4000002DA: FB 35 4F EC 35 A2 59 62 59 CF 11 CD 35 B6 98 A3 61 D9 59 9F AF 53	
0x4000002F4: 4C D3 1A 97 69 17 CF D7 36 5E 87 3E 9E 97 88 E5 BA 6C BC F1 9C BB D8 96 2E 96	
0x40000030E: F3 45 D7 36 37 71 DE CE F1 72 79 8D 98 C7 98 F8 DD 85 F3 D8 C7 71 7F 88 A7 C7	
0x400000328: C7 F8 97 1F 7E 8C 0F 4F 0F 71 1C BB 78 38 8C 11 5D C4 D0 75 D1 87 43 4C 2B BF	
0x400000342: CB 3E 5B 62 5D AF 7A F6 3C B3 BE ED A3 85 21 B6 DC BB 6B 5B F8 79 E6 81 EA A3	
0x40000035C: 8B DD 7D D0 BE 6D 3B EF 4F F6 43 C3 B9 90 D0 D8 62 C9 E7 CB 59 98 79 1F EC 91	
0x400000376: F6 DB AA 7B 5D E6 45 E7 6A 8D 2E B6 66 95 AD E9 DA 21 A2 99 A3 1B FA 6B 86 36	
0x400000390: FA 79 8D AE DF 7B AF B6 69 23 D6 D0 BE D7 73 F7 E9 8E 2D F7 8F EC A5 EC D0 9F	
0x4000003AA: 3F 59 3F 9D 75 ED 43 3E 7E D2 7D AC B9 EF F9 59 B0 6E 7E 6E D9 7F DE D3 7A FD	
0x4000003C4: 9A 7B 2F D6 D0 FE E6 8C AE 73 AC 2B 7F F2 5E E7 98 65 CB B1 0E 2C 9F 2C B6 CF	
0x4000003DE: 96 D6 04 C3 B7 D9 56 8E AD F3 6C 5B 34 9C BB C6 7B 9F BD 50 76 67 DB D6 98	
0x4000003F8: B9 36 EE 33 EC 07 F0 19 E7 F5 BE D6 FC 0E 1F B1 AD 53 74 DB CF 38 96 BF 69 86	
0x400000412: 58 5B 87 8E 4D 8E 8D E4 9E 79 0F EE B5 7C D0 1C 0B 96 63 5D A2 99 B1 89 EC 81	
0x40000042C: 5C 23 7D 7F 96 91 98 D6 6B 6C 8E 4B AE C5 1C 9B FE 2E 07 C5 B9 1A 74 CD 4D 83	
0x400000446: 05 5A 63 5E 16 1D 43 5F 53 27 1B A6 BE 9E B0 D7 73 CC 3C 13 F6 5A BB EA OC C8	
0x400000460: FF 6C 11 FD 1A D1 E1 9B 58 1C D6 0E 5F 21 FB 1B D1 35 11 4B 78 99 2E FD 31 D7	
0x40000047A: D5 76 98 DE 97 B3 BF E5 19 E7 BE B0 29 7A 66 98 BE FA 3B 6D 74 DA 9F 7A 36 F8	
0x400000494: 31 D9 10 1E A1 F7 5B DB F4 B1 CE 57 DB 47 19 D6 7C 13 1D CC 35 7D 6C AB DF C3	
0x4000004AE: 94 CA 2E 71 4D EC DA DB FE E3 48 B1 F6 3C 2B FE 24 7B 64 8E A5 C7 36 B5 B1 6E	

Memory 1	
Address: 0x40000036	
0x400000036: 78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B	
0x400000050: DF FF F3 3C B3 B5 DD 9D 6D 59 B2 02 40 AA 67 3F F7 08 80 B3 1C EB A9 2A 92 00	
0x40000006A: 32 4F 9E 13 C2 C3 A3 F9 F0 F1 B0 B5 7D 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35	
0x400000084: 86 B5 89 DD D0 C7 DC 4F D1 F6 40 B8 5D 18 CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B	
0x40000009E: 74 31 C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 E4 77 8B CB	
0x4000000B8: 14 D1 B6 31 34 6D BA BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB DB DA 55 EF D7	
0x4000000D2: 44 17 5D D3 46 34 FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F	
0x4000000EC: 6D DB 26 BA 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D 53 74	
0x400000106: 9D 5F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F	
0x400000120: 6B 22 86 46 F7 16 CD 18 FD CA 05 6C 31 F3 DE 4D E3 F5 8B 2E 5A 9B B1 3F 44 2C	
0x40000013A: 73 CC 5B C4 DC 46 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC D2 6B B7 45 D3	
0x400000154: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C	
0x40000016E: 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC 13 C3 03 6B BB B6 5A 1F	
0x400000188: 6E 5A 35 9A FB 26 FA 7B 8B 66 5C B5 E2 5B DF EA DF D0 7F 48 F4 55 EF D5 0F	
0x4000001A2: 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23 EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1	
0x4000001BC: 63 6D 87 D0 6D 3D 58 FB 86 D5 B3 D8 96 88 A1 4D 62 DB B1 3F 9C 05 61 3C C6	
0x4000001D6: AE 8B 6D 6B FA 7A 3D EB C6 F7 FA FE B9 2F EB A4 F5 72 7D 78 6F 99 F8 6D 56 FB	
0x4000001F0: 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 E1 96 78 5F BE C7 FE B8	
0x40000020A: F2 8A 6D 8B 31 06 7D 9F CB 9E F2 B3 F6 FB D7 F7 79 9E A1 E7 5E 6B CD 7D BE	
0x400000224: FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43 5D 0B DF E3 3D 16 36 42 5E D3	
0x40000023E: BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1 AD B1 B5 87 68 A2 8D 69 99 A2 E3 D8 2C 3C	
0x400000258: A3 41 F7 CB 3D F2 99 6D BB 8F 75 F1 07 D7 FA B1 F7 B5 BF B5 A6 5E 87 5F BE 62	
0x400000272: E5 01 6C BE 06 4D 88 75 8E 36 72 BF B1 6E 6C 78 2D 23 37 C4 46 D3 8E B1 C4 A4	
0x40000028C: 75 E2 87 88 1B BB 38 1C 77 B1 3F 34 F1 F5 F0 14 4F BB 0F B1 3F 9C 05 61 3C C6	
0x4000002A6: FE 61 17 8F BB 31 1E 0F FB 18 4E BB 38 EC 0F 71 1A 9E E2 B8 3F C5 E9 70 8A 71	
0x4000002C0: 38 C4 D0 8D DA BF FB F6 29 BA B1 1A 5D 3F 6B FF 75 ED 51 7B 6F D0 91 7B 89 A6	
0x4000002DA: FB 35 4F EC 35 A2 59 62 59 CF 11 CD 35 B6 98 A3 61 D9 59 9F AF 53	
0x4000002F4: 4C D3 1A 97 69 17 CF D7 36 5E 87 3E 9E 97 88 E5 BA 6C BC F1 9C BB D8 96 2E 96	
0x40000030E: F3 45 D7 36 37 71 DE CE F1 72 79 8D 98 C7 98 F8 DD 85 F3 D8 C7 71 7F 88 A7 C7	
0x400000328: C7 F8 97 1F 7E 8C 0F 4F 0F 71 1C BB 78 38 8C 11 5D C4 D0 75 D1 87 43 4C 2B BF	
0x400000342: CB 3E 5B 62 5D AF 7A F6 3C B3 BE ED A3 85 21 B6 DC BB 6B 5B F8 79 E6 81 EA A3	
0x40000035C: 8B DD 7D D0 BE 6D 3B EF 4F F6 43 C3 B9 90 D0 D8 62 C9 E7 CB 59 98 79 1F EC 91	
0x400000376: F6 DB AA 7B 5D E6 45 E7 6A 8D 2E B6 66 95 AD E9 DA 21 A2 99 A3 1B FA 6B 86 36	
0x400000390: FA 79 8D AE DF 7B AF B6 69 23 D6 D0 BE D7 73 F7 E9 8E 2D F7 8F EC A5 EC D0 9F	
0x4000003AA: 3F 59 3F 9D 75 ED 43 3E 7E D2 7D AC B9 EF F9 59 B0 6E 7E 6E D9 7F DE D3 7A FD	
0x4000003C4: 9A 7B 2F D6 D0 FE E6 8C AE 73 AC 2B 7F F2 5E E7 98 65 CB B1 0E 2C 9F 2C B6 CF	
0x4000003DE: 96 D6 04 C3 B7 D9 56 8E AD F3 6C 5B 34 9C BB C6 7B 9F BD 50 76 67 DB D6 98	
0x4000003F8: B9 36 EE 33 EC 07 F0 19 E7 F5 BE D6 FC 0E 1F B1 AD 53 74 DB CF 38 96 BF 69 86	
0x400000412: 58 5B 87 8E 4D 8E 8D E4 9E 79 0F EE B5 7C D0 1C 0B 96 63 5D A2 99 B1 89 EC 81	
0x40000042C: 5C 23 7D 7F 96 91 98 D6 6B 6C 8E 4B AE C5 1C 9B FE 2E 07 C5 B9 1A 74 CD 4D 83	
0x400000446: 05 5A 63 5E 16 1D 43 5F 53 27 1B A6 BE 9E B0 D7 73 CC 3C 13 F6 5A BB EA OC C8	
0x400000460: FF 6C 11 FD 1A D1 E1 9B 58 1C D6 0E 5F 21 FB 1B D1 35 11 4B 78 99 2E FD 31 D7	
0x40000047A: D5 76 98 DE 97 B3 BF E5 19 E7 BE B0 29 7A 66 98 BE FA 3B 6D 74 DA 9F 7A 36 F8	
0x400000494: 31 D9 10 1E A1 F7 5B DB F4 B1 CE 57 DB 47 19 D6 7C 13 1D CC 35 7D 6C AB DF C3	
0x4000004AE: 97 CA 2E 71 4D EC DA DB FE E3 48 B1 F6 3C 2B FE 24 7B 64 8E A5 C7 36 B5 B1 6E	

Memory 1	
Address: 0x41002580	
0x41002580: 5E 73 EC 08 8F EF 7B 3C 9D 02 3F B3 2A 4F C3 F0 7D C5	
0x41002592: B6 35 DD 4F 3A BA B1 B5 31 D1 32 B1 CD 46 CB 31 BC D4	
0x410025A4: 4B F4 55 17 34 17 8B 7A 13 7F BA E5 A2 FB C6 4D 9F F0	
0x410025B6: BA 89 69 59 3F 46 18 6C 4D 2E 3F CC 46 37 11 2F 12 D3	
0x410025C8: 8D E5 C4 2C C3 4E 6F E3 9E 18 CC 6B 1F 9A 76 B5 EA F0	
0x410025DA: EF 6C 45 AC 23 D0 E6 B1 D0 58 B3 A1 B1 D7 8B 7A F7 2F	
0x410025EC: E7 99 FB 58 E5 0F 76 96 C7 8A 06 CB F6 79 5E BE BE F5	
0x410025FE: 4B 43 E3 42 B2 CD E3 B1 A2 A2 3C CB 6D F1 B1 B5 B6 01	
0x41002610: 6D 36 6E 23 DA A4 88 1C 34 14 B1 61 61 31 18 EC 9E C5	
0x41002622: 71 8D F6 1A FF 7B 78 73 35 A2 CF B6 F9 D1 53 97 D7 3E E5	
0x41002634: F1 96 D5 71 72 C7 85 71 C7 1F 4F BB 11 75 4C 3E AF B3	
0x41002646: 89 B8 79 A3 D0 EF C3 D8 C8 1F DB E6 8D 95 21 1B 36 AE	
0x41002658: B6 D0 F7 F7 EC E9 ED D2 EF F6 7F FD D6 8C 2B E7 B1 2C	
0x4100266A: D6 D9 F3 34 7B 76 98 33 19 D6 B1 BD BF 58 8E 79 7C 96	
0x4100267C: 99 81 7F D6 4B 9B C5 CD 5A 1D 27 9E CC 5A 8C 7D 9B 0E	
0x4100268E: 1B 4D 76 B3 E7 7A F4 DA F8 1E DB 57 D6 CC AB CA EC	
0x410026A0: E3 3C 7B 67 C6 43 16 EF BD DD 26 96 E3 FE 2F 4B 58 1C	
0x410026B2: 8D 85 3E E9 62 5B 9D C7 B7 9F B4 97 96 FB CD CB DB B6	
0x410026C4: CF F4 6C 36 B0 FC 85 9B 8F 9B E3 53 72 C5 06 EA FE AE	
0x410026D6: 18 66 9D 32 3E A3 46 E5 38 F8 59 AB 04 97 DB C3 45 7D	
0x410026E8: FC 9D 6A 87 C6 EC B7 82 88 7C 77 7D E7 C6 FE 8A D8 1E C1	
0x410026FA: 79 31 FE 1E 46 A7 49 38 ED A5 96 CF CC FE 5D F5 01 FA	
0x4100270C: 6E 8F 8E E6 5E 7F 39 FA EF 23 1E 81 16 E1 CE DA 11 4B	
0x4100271E: D8 36 D6 8E 63 BB E9 C5 98 CE E2 EF EB E8 FE 1B 3D 33	
0x41002730: 68 8B 36 EB CB 57 97 65 ED C7 7C 66 8E 3F ED 35 F9 1B	
0x41002742: 13 06 9E 43 F9 B5 7A E9 BF 2D FB 31 4E B2 F3 F2 0E F8	
0x41002754: D9 1E 83 C8 F3 5B 6F C5 B1 F8 E3 6B E3 F1 8A 9C 26 F7	
0x41002766: 0F 89 4C DF 35 5F CF 47 03 9C 4A 28 40 F0 55 70 7D	
0x41002778: EE 69 B4 1E 44 A1 4E CB 64 77 72 2A D0 F5 F0 AE 18 34	
0x4100278A: F0 8A E1 FC 7C 4E 43 E5 28 25 C5 D7 E8 6E B1 D0 63 B6	
0x4100279C: 7B 3C 76 B5 08 1D 49 B4 D7 DC ED 3A 04 1B C6 4D 74 13	
0x410027AE: F4 B8 D0 2C D6 AD EC F4 04 80 F6 FB 70 19 BA 0C 3C 71	
0x410027C0: 97 12 FE A5 E8 E8 F7 20 C1 EB 9D 35 A5 71 3F 98 06 F8	
0x410027D2: F9 58 8C 9C 4E 13 07 3C 5B 67 3A 6A 0F 5D C6 31 3E 61	
0x410027E4: 63 76 3C 78 8C F6 16 78 FB E8 4E D7 34 B8 8A 6D B7 DB	
0x410027F6: EB 32 5C A5 28 92 FE D4 C6 F3 34 1D 78 CF E3 F4 21 08	
0x41002808: 03 C2 C6 64 03 65 3A 73 6D 9E 20 E5 FE 28 38 A7 1B F6	
0x4100281A: 8E 90 D3 67 9E 9F 87 3B BD 90 B0 4F 0E B4 2A FA 3C B1	

Address:	0x41004B00	
0x41004B00:	5C 23 1B B2 58 B5 DF B3 6D 40 F7 1C 92 9E C3 F1 17 D2	
0x41004B12:	6D 86 D0 D1 5D C5 B6 6D C6 C7 C7 B6 18 B4 14 34 47 6C	
0x41004B24:	C4 BB EF 5D FE 4D 6D E5 EB 6D 66 97 F5 03 D6 53 D0 CB	
0x41004B36:	DF 79 9A 7F 6B F7 FD 31 E3 5A 44 5B 6C 37 4B 5B 6B F8	
0x41004B48:	3E E7 D8 AC 30 D1 7F 7A 13 31 F3 BB D6 FB 8B EE DF 48	
0x41004B5A:	D5 6D D3 65 EB 4E E7 63 6E F9 D8 6D 9E 29 6D 3D FA BF	
0x41004B6C:	7D F9 6F 63 E0 3E F3 78 FE 6D 7D 9E FB 9E 6B FA 8B B2	
0x41004B7E:	CF 5D 3D 5E 87 7F 16 B5 8D E3 A3 3D BB 07 F7 A6 EB E2	
0x41004B90:	88 72 6C 37 8E 75 18 77 F1 4F 3F 3C 17 1E 4E E2 EF	
0x41004BA2:	38 DA 29 5D 75 6F 89 4F 59 11 98 5B B9 4C 69 36 9E BA	
0x41004BB4:	9C 2E F7 DE 79 98 F3 7F C7 7E 0F 78 5D D1 2B 5B 7A B6	
0x41004BC6:	21 6B E6 8B BE 4F B9 62 59 EC AA 45 2E AD A2 FA 9A D6	
0x41004BD8:	69 BE E9 8F 0D 3F 43 7D F9 7E DE 9A D0 AE 7F 98 0E B6	
0x41004BEA:	04 56 6C 9C 9F 67 B9 EC E7 FC AD CF 69 B7 8D 0F D0 63	
0x41004BFC:	B1 5C 96 6B AE F6 B9 4D 63 43 1B B0 3C BB FF 1A 58 5F	
0x41004C0E:	D1 7B 31 9B BF FE 66 3B 9F 31 A1 F4 D8 7C 35 DF E2 DA	
0x41004C20:	48 2B 64 36 6F 17 6D 1D C6 9D 2E 78 7C BE D7 1B E2 DF	
0x41004C32:	57 4F D8 E3 5E AC D7 B2 79 DF 4D 75 11 0C 9B 56 D8 AB	
0x41004C44:	98 5D 6B 31 DB FC 3C 9E 4F B5 9A 9C 17 8A 9D 7A 64 53	
0x41004C56:	EB F6 FC 73 DE 9D F7 F8 37 E2 1B D9 7B D9 79 26 34 8F	
0x41004C68:	7E DA D6 5B 6B 79 F8 6E 69 CE 58 B3 7C EF 7D E8 A7 71	
0x41004C7A:	1D 71 61 75 B1 DB B0 D6 AF D1 2D 97 3A A6 F8 6D 88 6E	
0x41004C8C:	8A B8 81 39 F5 1B E6 9E CB A6 E4 BC 9F E7 91 55 73 2C	
0x41004C9E:	16 7A 61 B9 18 46 1C 39 35 70 49 75 CA C6 9C E2 7F EE	
0x41004CB0:	FA E3 F9 C4 C4 CE 35 3B A2 2E 16 9B E1 C6 2E BA C6 62	
0x41004CC2:	F8 BD AE 17 67 95 E8 FB 67 5E 1E CF FB 71 F7 C7 BF 4C	
0x41004CD4:	04 EB BC F3 BE 78 FC 16 3F B0 10 3C F3 E7 87 B7 9F 0D	
0x41004CE6:	BA 31 D3 B7 E2 E2 E2 F1 F3 16 C1 C0 B6 8C 7A 7E 33 BD	
0x41004CF8:	38 85 E3 23 0F F8 6D 49 B1 7F 70 16 DC 7A 0A 95 65 71	
0x41004D0A:	72 B7 B0 B9 3A 0F 20 5E 3A E6 D8 71 3B 22 57 B5 EE 59	
0x41004D1C:	E5 73 0C 9F CE D8 FB 29 6E B4 60 8C 31 8A 30 5D C7 14	
0x41004D2E:	DA F7 A1 F3 07 84 78 32 FD E9 E7 86 1C D6 25 79 8C 5E	
0x41004D40:	81 11 BE E3 18 D0 0F 22 D9 3A 40 A8 A4 F0 05 6C 82 38	
0x41004D52:	3E 66 A8 B3 80 0C BF 92 37 15 0F E8 87 7C 87 74 7C 1F	
0x41004D64:	EC 0F 9C 88 E7 EF FB 8C 16 79 86 68 FD 46 43 DE 3D 26	
0x41004D76:	B5 C4 6D 8F E0 AF D8 C4 F9 DA B1 1E D8 9F A7 63 C6 68	
0x41004D88:	FC F9 EA C8 E4 5F F4 2D 5D 87 A3 A2 E2 09 0B AF BC F9	
0x41004D9A:	A4 8E 39 27 C4 AD 36 A4 92 14 1D 1B CB 7C C9 5C 91 16 CF 9D	
0x41004DAC:	C0 5C FE 21 C0 3E A6 84 11 D1 0E 8F 89 09 CA 01 E6 99	

Subproject 2 (작성자: 유지원)

- Assem.s

```
1      AREA    Reset, CODE, READONLY
2      ENTRY
3      EXPORT  rgb_negative
4
5      rgb_negative
6
7      LDR     r0, =0x40000036      ; input image start address (RGBA)
8      LDR     r1, =0x20000000      ; output image start address (RGB)
9      MOV     r2, #9600            ; total 9,600 pixels to process
10
11     loop
12         ; Read and invert R
13         LDRB  r3, [r0], #1        ; r3 = R, r0 += 1
14         RSBS  r3, r3, #255        ; r3 = 255 - R
15
16         ; Read and invert G
17         LDRB  r4, [r0], #1        ; r4 = G, r0 += 1
18         RSBS  r4, r4, #255        ; r4 = 255 - G
19
20         ; Read and invert B
21         LDRB  r5, [r0], #1        ; r5 = B, r0 += 1
22         RSBS  r5, r5, #255        ; r5 = 255 - B
23
24         ; Skip Alpha (just move pointer 1 byte forward)
25         ADD   r0, r0, #1
26
27         ; Store inverted RGB values (3 bytes per pixel)
28         STRB  r3, [r1], #1        ; store R
29         STRB  r4, [r1], #1        ; store G
30         STRB  r5, [r1], #1        ; store B
31
32         ; Loop control
33         SUBS  r2, r2, #1
34         BNE   loop
35
36         BX   lr                  ; return
37
38     END
```

: 이 코드는 32비트 RGBA 이미지 데이터를 입력으로 받아, 각 픽셀의 Red, Green, Blue 값을 색 반전한 뒤 24비트 RGB 이미지 형태로 출력하는 기능을 수행한다. 각 픽셀은 4바이트(RGBA)로 구성되어 있으며, 색 반전은 각 색상 성분에 대해 255에서 해당 값을 빼는 방식으로 구현된다. 이 프로그램은 총 9,600개의 픽셀에 대해 이러한 변환을 수행한다.

프로그램의 진입점은 rgb_negative 함수이며, r0 레지스터는 입력 이미지의 시작 주소인 0x40000036으로 설정되고, r1 레지스터는 출력 이미지의 시작 주소인 0x20000000으로 설정된다. r2 레지스터는 루프 카운터로 사용되며, 총 9,600번 반복된다.

루프 내부에서는 먼저 LDRB 명령어를 통해 1바이트씩 Red, Green, Blue 값을 차

례대로 읽어온다. 각 색상 값은 RSBS 명령어를 사용하여 255에서 해당 값을 뺀 결과로 색 반전된다. Alpha 값은 사용하지 않기 때문에 별도로 읽지 않고 ADD 명령어를 통해 입력 포인터 r0만 1바이트 증가시켜 건너뛴다.

반전된 R, G, B 값은 각각 STRB 명령어를 통해 출력 주소 r1에 순차적으로 저장되며, 저장 후에는 r1이 1바이트씩 증가된다. 이 과정을 통해 입력의 4바이트 (RGBA) 픽셀은 출력으로 3바이트(RGB) 형태로 재구성된다.

루프가 한 번 돌 때마다 r2를 1씩 감소시키고, BNE 명령어를 통해 r2가 0이 아닐 경우 다시 루프를 반복한다. 9,600번의 반복이 끝나면 BX lr 명령어를 통해 함수가 종료된다.

이 코드는 입력 이미지를 손상시키지 않으면서, 별도의 출력 공간에 색 반전된 결과를 순차적으로 저장하며, 알파 채널을 무시하고 RGB 정보만을 다루는 점에서 24비트 RGB 변환 로직에 초점을 맞춘 구조로 되어 있다. 또한, 반복 구조와 포인터 증가는 모두 바이트 단위로 처리되어 RGBA 형식에 정확하게 대응되고, 각 레지스터는 목적에 따라 명확하게 분리되어 사용되고 있어 ARM 어셈블리 코드로서 구조적 안정성과 실행 효율성을 모두 고려했다.

- 결과

Memory 1	
Address:	0x40000036
0x40000036:	78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B DF
0x40000051:	FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F F7 08 80 B3 1C EB A9 2A 92 00 32 4F
0x4000006C:	9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89
0x40000087:	DD D0 C7 DC 4F D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31 C6 DA
0x400000A2:	2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 B4 E7 88 CB 14 D1 B6 31 34
0x400000BD:	6D B4 BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB D8 DA 55 EF D7 44 17 5D D3 46 34
0x400000D8:	FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F 6D DB 26 BA 66 8B 79
0x400000F3:	E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D 53 74 0D 9F D0 47 B3 F0 CB 4D
0x4000010E:	CC BA DF 8E 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD
0x40000129:	CA 05 6C 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46 6C 7D 44 37
0x40000144:	37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3 F6 B1 6C 8D 3E 87 7B E5 E7 DB BC
0x4000015F:	C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F
0x4000017A:	D3 18 31 0E 6D CC F3 12 03 6B BB B6 5A 1F D6 9A 35 9A FB 26 FA 76 8B 66 5C B5 EE
0x40000195:	5B DF EA DF 5D D7 F0 48 F4 55 EF D5 0F 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23
0x400001B0:	EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1 63 6D 87 D0 6E 60 3D 58 F9 B6 D5 B3 D8 96 88
0x400001CB:	A1 6D 62 DB B1 9E 4B 2C D7 29 BA AE 8B 6D 6B F4 7A 3D EB C6 F7 FA FE B9 2F EB A4
0x400001E6:	F5 E7 7D F8 6F 99 F9 BD 56 FB 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3
0x40000201:	73 E1 96 78 5F BE C7 FE B8 F2 8A 6D 8B 31 06 7D DF 9F CB 9E F2 B3 F6 FB D7 F7 79
0x4000021C:	9E A1 E7 E5 6B CD 7D BE FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43 5D 0B DF
0x40000237:	E3 3D 16 36 42 5E D3 BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1 AD B1 B5 87 68 A2 8D 69
0x40000252:	99 A2 E3 D8 2C 3C A3 41 F7 CB 3D F2 99 6D BB 8F 75 F1 07 D7 FA B1 F7 B5 BF B5 A6
0x4000026D:	5E 87 B6 FB 62 F5 01 6C BF 06 6D 88 75 8F 36 72 BF B1 6F 6C 78 2D 23 37 DC 46 D3

Registers	
Register	Value
Current	
R0	0x40003401
R1	0x200026D6
R2	0x0000188E
R3	0x00000099
R4	0x00000036
R5	0x00000004
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R1...	0x00000000
R1...	0x00000000
R1...	0x00000020
CP...	0x20000003
SP...	0x00000000
User/...	

Memory 1																	
Address: 0x40000036																	
0x400000036:	78	5E	5C	BD	77	73	23	59	92	EC	1B	29	21	08	B2	44	77 8F 58 71 67 EF
0x40000004C:	B5	77	FF	7B	DF	FF	F3	3C	B3	B5	DD	9D	6D	59	82	02	40 AA 67 3F F7 08
0x400000062:	80	B3	1C	EB	A9	2A	92	00	32	4F	9E	13	C2	C3	C3	A3	F9 F0 F1 B8 B5 7D
0x400000078:	17	73	33	C5	D2	75	D1	B6	6D	6C	EB	35	86	B5	89	DD	D0 C7 DC 4F D1 F6
0x40000008E:	4D	34	5D	1B	CD	BA	C5	D6	6C	B1	B6	5B	F0	B5	6D	5B	74 31 C6 DA 2C D1
0x4000000A4:	C7	16	CD	32	C7	D2	AE	B1	B6	5D	B4	CD	18	6B	DB	46	B4 E7 88 CB 14 D1
0x4000000BA:	B6	31	34	6D	B4	BC	47	D7	C6	D4	6C	31	AD	4B	C4	14	31 F4 BB D8 DA 55
0x4000000D0:	EF	D7	44	17	5D	D3	46	34	FE	37	EF	17	4D	A3	CF	8B	6D 0B 3E 7A E5 9F
0x4000000E6:	4D	13	EB	BA	06	7F	6D	DB	26	BA	66	8B	79	E5	97	9A	D8 A2 F5 F5 F0 FB
0x4000000FC:	03	BF	B1	C6	D6	74	31	4D	53	74	0D	9F	D0	47	B3	F0	CB 4D CC BA DF 8E
0x400000112:	5F	89	79	5A	A3	69	9A	98	E7	59	7F	AE	2B	3F	6B	22	86 46 F7 16 CD 18
0x400000128:	FD	CA	05	6C	31	F3	DE	4D	E3	F5	B8	2E	5A	9B	B1	3F	44 2C 73 CC 5B C4
0x40000013E:	DC	46	6C	7D	44	37	37	D1	6D	11	4B	13	D1	2F	5B	CC	CD 12 6B B7 45 D3
0x400000154:	F6	B1	6C	8D	3E	87	7B	E5	E7	DB	BC	C4	D8	36	B1	2C	AC 65 13 C3 30 C4
0x40000016A:	D6	4E	D1	6C	9D	6F	7F	DA	A2	E3	7A	AF	B3	9E	13	5F	D3 18 31 0E 6D CC
0x400000180:	F3	12	03	6B	BB	B6	5A	1F	D6	9A	35	9A	FB	26	FA	76	8B 66 5C B5 EE 5B
0x400000196:	DF	EA	DF	5D	D7	F0	48	F4	55	EF	D5	0F	4D	6C	6D	C3	6A 45 D3 2C FA AC
0x4000001AC:	65	19	74	23	EB	36	C7	D0	4E	D1	C6	E6	E7	B2	69	B1	63 6D 87 D0 6E 60
0x4000001C2:	3D	58	F9	B6	D5	B3	D8	96	88	A1	6D	62	DB	B1	9E	4B	2C D7 29 BA AE 8B
0x4000001D8:	6D	6B	F4	7A	3D	EB	C6	F7	FA	FE	B9	2F	EB	A4	F5	E7	7D F8 6F 99 F9 BD
0x4000001EE:	56	FB	6F	DE	96	58	63	89	86	E5	E0	5E	A3	0F	3E	48	7B 76 F3 73 E1 96

Memory 1																	
Address: 0xE2000000																	
0x200000000:	87	A1	A3	88	8C	DC	6D	13	E4	DE	F7	4D	88	70	A7	98	10 4A 00 84 20 0C
0x200000016:	C3	4C	22	62	92	7D	FD	BF	98	C0	08	7F	4C	E3	56	D5	6D CD B0 61 3D 3C
0x20000002C:	3C	06	0F	0E	4A	82	E8	CC	3A	2D	2E	49	92	14	CA	79	76 22 2F 23 B0 2E
0x200000042:	B2	CB	A2	32	45	3A	93	4E	49	0F	4A	92	8B	CE	39	D3	2E 38 32 CD 38 51
0x200000058:	4E	49	4B	32	E7	24	B9	4B	77	34	EB	49	CE	CB	4B	43	B8 39 2B 93 52 B4
0x20000006E:	3B	CE	0B	44	25	AA	10	BB	E8	A2	B9	CB	01	10	E8	B2	30 74 92 C1 85 1A
0x200000084:	B2	EC	14	F9	80	92	D9	45	99	86	1A	68	27	5D	0A	0F	04 FC 4E 39 29 CE
0x20000009A:	B2	AC	F2	60	2F	4C	0F	34	33	45	20	A0	76	86	5C	96	65 18 A6 80 D4 C0
0x2000000B0:	94	79	B9	08	32	E7	02	FA	93	CE	21	B2	1C	47	D1	A5	4E C0 BB 8C 33 A4
0x2000000C6:	23	B9	93	BB	C8	C8	92	EE	B4	2E	D0	A4	32	ED	94	BA	2C 09 93 72 C1 84
0x2000000DC:	1A	18	43	3B	27	4E	D3	53	EC	3C	CF	29	B1	2E	62	90	80 5D 1C 85 4C 61
0x2000000F2:	EC	2C	E7	CE	92	33	0C	FC	94	44	A5	E0	29	CA	65	04	05 89 74 A3 4A 11
0x200000108:	20	15	20	28	0F	B7	AA	10	2A	B2	93	92	95	BA	2C	05	53 9A 8B DC 14 38
0x20000011E:	2F	B1	39	19	18	96	4E	9C	78	2F	91	C2	A7	06	2A	4C	27 77 5E 92 24 4E
0x200000134:	61	D3	28	D6	51	74	92	0B	85	C2	39	08	05	46	D0	14	0A 18 82 90 66 06
0x20000014A:	A9	04	90	69	A7	9C	79	1A	1F	5C	F0	C1	84	89	0C	1E	69 87 41 38 01 0D
0x200000160:	75	92	CE	F9	82	60	34	61	4C	09	04	08	86	61	18	1A	94 82 41 05 05 41
0x200000176:	74	0A	0A	4D	14	B4	30	30	BC	A2	20	1C	C2	C9	BD	A1	45 4D 78 48 32 80
0x20000018C:	A3	1C	E9	52	4E	4A	97	5D	72	66	5D	1C	D3	C3	5C	08	34 C2 66 92 44 8A
0x2000001A2:	0E	F8	05	4E	08	40	4A	59	78	49	14	1A	FE	93	F9	92	77 71 C9 8D 4E 91
0x2000001B8:	93	D2	DC	C8	B9	2C	71	3B	5B	8A	78	77	E7	C7	E3	88	C0 CB 0E 0F EB B0

Subproject 3 (작성자: 김선영)

- Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT gray_scale ; 3*R + 6*G +1*B
3
4      ENTRY
5
6
7
8      gray_scale
9          LDR    r0, start
10         LDR    r1, pixel
11         MOV    r2, #9600
12
13     loop
14         LDRB    r3, [r0], #1      ; R값
15         ADD     r3, r3, r3        ; r3=2*R
16         ADD     r3, r3, r3, LSR #1 ; r3=3*R
17
18         LDRB    r4, [r0], #1      ; G값
19         LSL     r4, r4, #2        ; r4=4*G
20         ADD     r4, r4, r4, LSR #1 ; r4=6*G
21
22         LDRB    r5, [r0], #2      ; B값 (A 사용 X)
23
24         ADD     r6, r3, r4        ; r6= 3*R + 6*G
25         ADD     r7, r6, r5        ; r7= 3*R + 6*G +B
26
27         STRH    r7, [r1], #2      ; r7 값 r1에 저장, r2 2B 증가
28
29         SUBS    r2, r2, #1
30
31         BNE     loop
32     start DCD 0x40000036          ; RGBA 픽셀 데이터 시작 주소
33     pixel DCD 0x20000000          ; Grayscale 결과 저장 주소
34
35     END
```

: 이 함수는 RGB(A) 형식의 픽셀 데이터를 입력으로 받아 각 픽셀의 회색조 값을 계산한 후 메모리에 저장한다. 픽셀 데이터는 start 레이블에 지정된 시작 주소에서 불러온다. 각 픽셀은 R, G, B, A 순서로 구성되어 있으며 이 중 A 값은 사용하지 않고 R, G, B 값만 활용한다. 각 픽셀에 대해 회색조 변환 공식 $3R + 6G + B$ 곱하기 R 더하기 6 곱하기 G 더하기 B 를 적용한다. 계산된 회색조 값은 16 비트 단위로 결과를 저장할 주소에 기록한다. 총 9600 개의 픽셀에 대해 이 작업을 반복한다. R, G, B 값은 각각 1 바이트씩 읽으며 A 값은 건너뛰고 다음 픽셀로 이동한다. 루프는 SUBS 명령어로 제어하여 픽셀 수만큼 반복한 뒤 종료한다.

- 결과

Address: 0x20000000	Address: 0x40000000
0x20000000: 6B 12 00 46 01 01 04 7C 21 3A 57 74 24 5C 8C 5A 22 7B 64 B8	0x40000000: 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 00 00 03 C0
0x20000004: EF C9 6D 75 7F 93 49 6A CE D3 45 A3 AB 5D D1 A1 74 A1 A5 67	0x40000004: 00 00 02 80 08 06 00 00 00 53 F9 11 43 00 00 00 01 73 52 47
0x20000008: 83 71 C0 89 79 B7 72 74 8F 6F 6C CA 99 8C 9A 8E 9C 99 78 80	0x40000008: 42 00 AE CE 1C E9 00 00 20 00 49 44 41 54 78 5E 5C 5D 77 73
0x2000000C: 78 B7 F2 61 83 40 8B 8C 9F 7E BF 6C 54 86 81 92 D6 85 4F 9B	0x4000000C: 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B
0x20000010: 60 84 72 94 7E B1 62 D1 87 5F 83 9E AC 96 73 3D 7B 8A 85 90	0x40000010: DF FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F 77 08 80 B3
0x20000014: 73 A4 9E 9E 72 96 93 46 90 8F A8 75 55 D7 B0 9E 65 71 AF 94	0x40000014: 1C EB A9 2A 92 00 32 4F 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D
0x20000018: DB CC B3 A7 A6 76 9F 5C B3 8E DB 5E AE C8 E9 C2 9C 8C C0 B0	0x40000018: 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89 DD D0 C7 DC 4F
0x2000001C: 7F 75 29 98 87 6D 63 8E 81 87 9C 97 98 80 D6 82 A6 62 81 98	0x4000001C: D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31
0x20000020: 52 89 A9 B1 69 78 F4 65 80 9D 69 85 79 40 90 AC 84 DD 85 55	0x40000020: C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46
0x20000024: 9F 9F C2 90 5D 6F 82 7D AC 76 73 8E 93 C0 BA 90 96 DF 88 C8	0x40000024: B4 E7 88 CB 14 D1 B6 31 34 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B
0x20000028: DD 93 C7 4D 40 62 96 8A 7B 5E 99 C8 7E B1 B5 AA 7C 9C B6 A4	0x40000028: C4 14 31 F4 BB D8 DA 55 EF D7 44 17 DD D3 46 34 FE 37 EF 17
0x2000002C: 7A C3 83 96 72 CB 9E B1 89 AD 7C A5 8B BD 56 6E 5E 9B A9 A3	0x4000002C: 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F 6D DB 26 BA
0x20000030: AC 64 E7 90 A8 93 01 00 00 00 00 00 00 92 7B D9 85 80 83	0x40000030: 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D
0x20000034: EF 02 B8 7E 6D 7F BA 50 94 AB 70 92 69 44 3C 51 83 B8 84 7B	0x40000034: 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69
0x20000038: 98 7D 79 8E 71 5B 84 BA 7F 93 92 57 6A 74 A9 D2 7A 47 59 BB	0x40000038: 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD CA 05 6C
0x2000003C: 4F EC 9F 8D 84 75 41 D1 71 6B 81 CC 63 85 6F CA E7 8C E0 AE	0x4000003C: 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46
0x20000040: 8C A1 CE 8F 5F B6 86 87 7E 96 6D 6C 41 3C 7A 72 7E 86 B9 AC	0x40000040: 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3
0x20000044: BD 72 C7 88 66 99 8E A0 C5 6B 79 B2 C9 52 96 98 8E D4 B0 88	0x40000044: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3
0x20000048: A4 C2 8F B4 8A 9D 7C 92 A0 7F 7D 7A 90 B4 A2 8D B0 7B 9B A5	0x40000048: 30 C4 D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18
0x2000004C: A2 8E 9C B7 A6 A6 82 7A 75 A8 B7 91 75 9A A4 6C 6C B4 C9 9F	

Address: 0x200257A0	Register	Value
	Current	
0x200257A0: BC 6B 24 64 59 BA B7 43	R0	0x40000037
0x200257A8: 7A 73 9A 5B 8B E7 9D 90	R1	0x41000001
0x200257B0: C9 60 74 CC 9B A5 9A 83	R2	0x41002580
0x200257B8: 84 4D 76 70 C6 7B 3C 94	R3	0x41004B00
0x200257C0: 94 81 98 3D 81 7E 95 9D	R4	0x00002580
0x200257C8: 82 DC 6F 3C B7 84 7E 77	R5	0x00000078
0x200257D0: 63 B7 80 6C 89 A6 33 7F	R6	0x00000000
0x200257D8: 69 AD 8A B4 2A 7E B2 72	R7	0x00000000
0x200257E0: 4B 9E 91 45 6A 60 92 7D	R8	0x00000000
0x200257E8: 4B A2 77 80 7E B0 84 8D	R9	0x00000000
0x200257F0: AB A2 AE 94 81 92 95 49	R10	0x00000000
0x200257F8: 2E 5F 8E 94 B3 1C 4A 6A	R11	0x00000000
0x20025800: 00 00 00 00 00 00 00 00	R12	0x00000000
0x20025808: 00 00 00 00 00 00 00 00	R13 (SP)	0x00000000
	R14 (LR)	0x00000004
	R15 (PC)	0x00000024
	CPSR	0x00000003
	SPSR	0x00000000

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

· Memory relocation (32-bit RGBA pixels)

-> {8-bit R}, {8-bit G}, {8-bit B}: RGBA에서 A(alpha)만 제외하고 24-bit R, G, B 따로 저장했을 때 어떤 특이점이 생기는지 서술

이미지 hex 파일의 색상 채널을 메모리에 재배치하는 방식에는 플래너 방식과 인터리브드 방식이 있다. 플래너 방식은 색상 채널을 분리된 영역에 저장하는 방식이고 인터리브드 방식은 색상 채널을 한 픽셀 단위로 연속으로 저장하는 방식이다. 플래너 방식은 특정 채널만 처리하거나 압축할 때 유리하다. 하지만 메모리 접근이 복잡하고 병렬 처리가 어렵다. 인터리브드 방식은 메모리 접근이 직관적이고 대부분의 디스플레이 장치와 호환된다. 하지만 채널 분리가 어렵다.

이번 실험에서 이미지 hex 파일을 alpha 채널을 제거한 뒤 메모리에 재배치하여 이미지 변환 함수를 동작 시키려고 하는데 RGB 채널을 인터리브드 방식으로 재배치하면 특이점이 생긴다. Alpha 채널을 제거하고 인터리브드 방식으로 relocation할 경우, 각 픽셀의 크기가 4바이트에서 3바이트로 줄어들게 된다. 이는 메모리 공간의 절감과 반복 loop에서 주소 증가량이 4에서 3으로 줄어든다는 결과를 낳아 메모리 접근량 감소 및 loop 구조의 단순화에 따른 실행 시간 감소를 가져올 것 같지만 연산의 정확도에는 영향을 주지 않아, 성능 개선이 미미하다. ARM architecture는 4바이트의 정렬(aligned access)을 선호하기 때문에, 3바이트 단위로 접근할 시에 메모리 접근에서의 penalty가 발생할 수 있기 때문이다. 실제로 RGB를 인터리브드 방식으로 메모리에 재배치하여 실험을 수행한 결과 성능 개선도 저조하고 몇 가지 오류들이 발생했다. ARM 코드 최적화에 중점을 뒀을 때 메모리 재배치는 모두 플래너 방식으로 진행했다.

Subproject 1 (작성자: 최은지)

- Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL Red_Pixel_Count_Re
7
8      relocate
9          LDR    r0, =0x40000036 ; r0 = RGBA 데이터 시작 주소
10         LDR    r1, =0x41000000 ; r1 = R값 저장할 목적지 주소
11         LDR    r2, =0x41002580 ; r2 = G값 저장할 목적지 주소
12         LDR    r3, =0x41004B00 ; r3 = B값 저장할 목적지 주소
13         MOV    r4, #9600 ; r4 = 픽셀 수
14
15      loop_relocate
16          LDRB   r5, [r0], #1 ;
17          STRB   r5, [r1], #1 ; R 저장
18
19          LDRB   r5, [r0], #1 ;
20          STRB   r5, [r2], #1 ; G 저장
21
22          LDRB   r5, [r0], #1 ;
23          STRB   r5, [r3], #1 ; B 저장
24
25          ADD    r0, r0, #1 ; A 무시
26
27          SUBS   r4, r4, #1
28
29          BNE    loop_relocate
30          BX     lr ; 재귀할 문
31
32
33
34      Red_Pixel_Count_Re
35          LDR    r0, =0x41000000
36          MOV    r1, #9600 ; r1: 총 픽셀 수_9600개
37          MOV    r2, #0 ; r2: 조건 만족하는 픽셀 수 count
38          MOV    r3, #0 ; r3: 현재 인덱스
39
40      loop
41          CMP    r3, r1
42          BEQ    done
43
44          LDRB   r4, [r0]
45          CMP    r4, #128
46          BLT    skip
47          ADD    r2, r2, #1
48
49      skip
50          ADD    r0, r0, #1
51          ADD    r3, r3, #1
52          B      loop
53
54      done
55          MOV    r0, r2
56          BX     lr
57
58      END
```

: 이 코드는 각각 RGBA 이미지 데이터를 RGB로 분리 저장하는 작업과, Red 성분이 128 이상인 픽셀 수를 세는 작업을 수행한다.

첫 번째 함수 relocate는 원본 RGBA 이미지 데이터가 저장된

메모리(0x40000036)에서 각 색상 성분(R, G, B)을 분리하여 별도의 메모리

공간으로 재배치한다. R 성분은 0x41000000, G 성분은 0x41002580, B 성분은

0x41004B00 주소에 저장된다. 각 픽셀은 4 바이트(R, G, B, A)로 구성되어 있으며,

이 루프에서는 R, G, B를 차례대로 저장하고 A는 건너뛴다. 총 9600개의 픽셀에

대해 이 작업을 반복한다. LDRB 와 STRB 명령어를 이용하여 바이트 단위로 데이터를 읽고 쓴다.

두 번째 함수 Red_Pixel_Count_Re 는 재배치된 R 성분만 저장된 메모리(0x41000000)를 순차적으로 읽으며, 각 값이 128 이상인 경우를 검사하여 조건을 만족하는 픽셀 수를 세는 작업을 수행한다. 루프는 9600 번 반복되며, R 값(r4)을 메모리에서 읽은 후 128 과 비교하고, 조건을 만족하면 r2 를 증가시킨다. 루프가 종료되면 최종적으로 조건을 만족한 픽셀 수를 r0 에 저장하고 함수가 종료된다.

- 결과

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x0000007C
CPSR	0x60000003
SPSR	0x00000000

R0 register 값 = red 값이 128 이상인 pixel 의 수

: 0x12C3=4803

3 에서 계산한 값과 같음을 알 수 있다.

- 함수 수행 전 후의 memory map 정보

[illegible]

Memory 1

Address: 0x41000000

```

0x41000000: 78 77 92 21 77 67 FF F3 D0 82 67 08 A9 32 C2 F9 B5 33 D1 B8 09 DC 7D 0C BD 31 F0
0x41000001: 74 2C CD AE B4 DB 88 BE 66 CA AD 31 DA 44 46 EF CF 3E 4C 06 26 79 D8 F0 CB C1 61
0x41000003: 0D B3 CF 5F A3 87 2B 86 CE 08 BE B1 73 DC 44 40 D1 CD 4C 6C 7B CB B1 13 D0 F4
0x41000004: 0A 03 B3 D3 4D 03 5A B5 F3 5C D7 55 4D 4A F7 74 C7 CA 68 97 3D 58 08 28 21
0x41000005: AE F4 C6 B9 55 67 56 96 86 A3 7B 1E BE F2 31 9F B7 87 7D FA 75 EBC F3 DF 3F
0x41000008: BA BE 5C AD 68 92 3D 97 F5 75 FA BE 87 85 06 B8 12 D1 46 4C 87 3B 3F F0 CF 05
0x41000009: FB FB FC BD 1A 3F 6A D0 F7 E1 AB 61 91 F8 35 59 35 61 A5 AF 1A CF 87 B8 BC BF
0x4100000B: F3 F1 98 1D CD C7 97 07 1F C8 D4 43 CB 5D 3C A3 DC F9 EA 78 4C 43 ED 7B 79
0x4100000D: F6 5D 6A 66 DA A3 B6 6D AF D6 73 2D A5 3F 75 7E B9 3D 74 2F 4E BC AD 58 E7 9F
0x4100000E: 86 87 AB 56 CE 50 D6 EE F0 BE F1 74 96 58 4D 9B 05 B8 AC AE EC 7D 98 EB 1C D7 74
0x41000010: 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
0x41000011: 4D FE FE 24 A5 B1 13 23 B8 71 F3 93 98 62 07 F6 FE CB 7C D2 F4 AE EC EF 0F
0x41000013: C1 D7 D6 1B 4D 5F DC 60 2C 37 05 25 BE 3F B9 4B ED 14 08 F6 CD AD 90 7C DB
0x41000015: D4 FA 2A 3D 43 D9 6B 06 2B FB ED 9F C1 08 4B 51 E1 DB EC ED CA 6A 40 E7
0x41000016C: CA 3D D1 E2 27 3C 95 79 6D 06 ED E3 B6 3A CE C1 E8 96 7B DE D8 9E 3E 7E 9D
0x41000018: 68 D8 74 95 68 9C D9 5E 76 70 7A 74 F3 22 95 5F F5 9C 7F 2F BD 4C 95 95 98
0x4100001DA: 0E 27 98 E8 B0 D9 9A AB 21 88 3C 21 F9 59 D9 6D AB B6 D9 4D 3C B6 D8 B3 C1
0x4100001D4: 57 9F BF 6A ED D5 10 36 EB 3A B3 93 93 93 93 93 93 93 93 93 93 93 93 93
0x4100001D5: 52 EA 5C 67 0B B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6 B6
0x4100001EE: 3A D3 16 C5 91 EB 5A 38 FC D0 30 7D 60 9F 86 F4 1E 3B DD 7B 38 52 DC 1C 31
0x410000208: B7 AD 50 20 1E 65 1C 82 CA 2A 3C 1A 99 74 B8 4F 87 58 80 12 DB FB C1 A2
0x410000222: F7 FA 9A 22 CF 39 C1 69 BE 31 6C 1 6B 04 11 9D F7 75 CD FA 4E A3 B9 5A 00
0x410000233: 7F 4E EE 0F 39 A1 3A 42 EB 1E 45 BA B3 10 77 25 F0 32 D0 05 27 BB 08 08
0x410000256: 56 35 31 7B 77 9F 0D F0 0E 0F 4F 38 D3 FB 36 71 70 72 01 04 44 D0 FB DB
0x410000270: 73 08 62 1F B5 5A 5F 13 E7 BE 4D 0D DB D1 BA 86 06 07 E2 C7 38 77 09 E9
0x410000284: 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A 0A
0x4100002A: 71 BC F3 BE 77 C2 CE 9C 5D C1 49 B1 D4 FC 66 35 10 5D 90 88 47 37 A2 27 83 B8
0x4100002BE: 3C 68 12 C8 E6 1E 7C 5C 5A 1D 10 AA 49 D0 C2 D9 CA 44 99 88 83 9A 7B 1 6A
0x4100002D8: BC AF 97 74 D0 70 9F 41 00 82 42 78 84 19 46 53 9C CF 45 C3 D7 FB D7 FA 80
0x4100002F2: 4E D6 1B 95 51 00 0B B5 FF 49 21 00 94 5A 70 96 C1 A2 0F BC 3C D2 DC 00 6B BF
0x41000030: 3C 67 37 C9 8B 00 1 E3 0D 0D 7E EC C4 B5 67 31 D7 20 94 86 5E 8C B3 B8
0x410000326: 18 7A 6A CE 05 27 9F 9C 78 6C E2 E0 DC A2 BB CC 3F F7 40 59 F2 00 42 D6
0x41000033A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x41000035A: AC 00 B0 22 35 F9 AE 09 27 00 CF CD F2 4A 80 38 21 AA 08 AB 43 04 32 D6 4D
0x410000374: 92 ED 93 74 20 9F 4D 08 F5 8A 12 30 AF 18 76 0D C0 FD 81 0C FF 92 4C 00 45 CE
0x41000038E: AF 97 FF FF 72 B9 9C F0 D8 61 FB BE BF BE F7 FC B6 A3 47 0C D5 12 4B AC 7A B1
0x4100003A5: 3A 9D 67 84 A4 BA AC C3 90 C6 EF 5A 8A C3 AA 0B 37 D4 1C 3A D6 C7 1E 79 12 62
0x4100003C2: 0A 88 DD DD 0A 64 7D A1 AE 98 98 08 2B 84 A4 A4 88 50 97 18 50 33 46 AA 67
0x4100003D: 04 44 12 8D A1 77 7A 17 10 8D B1 61 04 A4 91 D9 69 93 49 4F 85 25 54 B7

```

Subproject 2 (작성자: 유지원)

- Assem.s

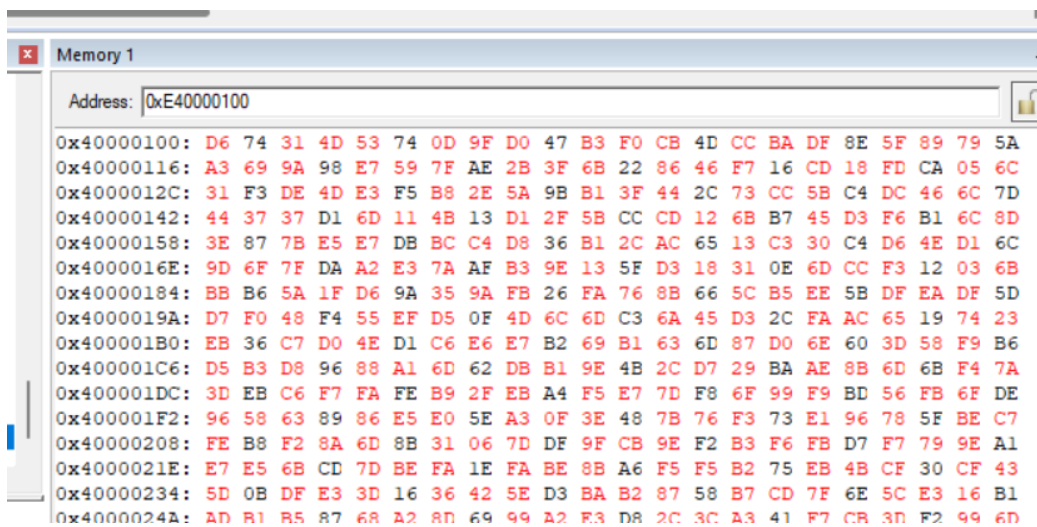
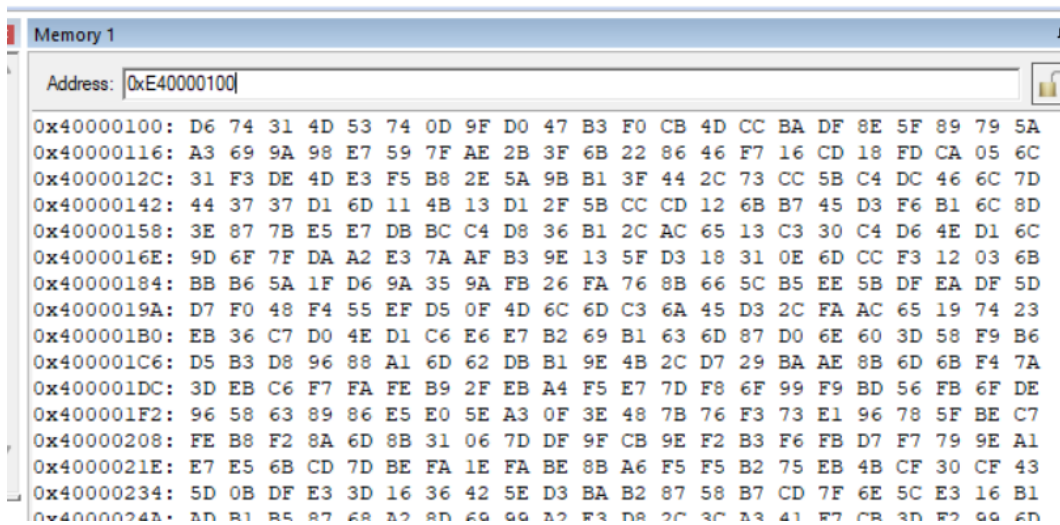
```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB   r5, [r0], #1
17         STRB   r5, [r1], #1
18
19         LDRB   r5, [r0], #1
20         STRB   r5, [r2], #1
21
22         LDRB   r5, [r0], #1
23         STRB   r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS   r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #9600
36
37     loop_negative
38         LDRB   r5, [r0], #1
39         RSBS   r5, r5, #255
40         STRB   r5, [r3], #1
41
42         LDRB   r6, [r1], #1
43         RSBS   r6, r6, #255
44         STRB   r6, [r3], #1
45
46         LDRB   r7, [r2], #1
47         RSBS   r7, r7, #255
48         STRB   r7, [r3], #1
49
50         SUBS   r4, r4, #1
51         BNE    loop_negative
52
53         BX     lr
54
55     END
```

: 이 코드는 32 비트 RGBA 형식으로 저장된 이미지 데이터를 처리하여, 색상 정보를 분리하고, 각 채널에 대해 색 반전 연산을 수행한 뒤, 최종적으로 24 비트 RGB 이미지로 재구성하는 기능을 수행한다. 총 9,600 개의 픽셀을 대상으로 동작하며, 두 개의 주요 함수인 relocate 와 re_rgb_negative 로 구성되어 있다. 프로그램의 시작점인 start 에서는 두 함수를 차례대로 호출한다.

먼저 relocate 함수는 입력 이미지의 시작 주소인 0x40000036 에서부터 4 바이트씩 데이터를 읽어들인다. 각 픽셀은 4 바이트로 구성되어 있으며, Red, Green, Blue, Alpha 순서로 저장되어 있다. 이 함수는 각 픽셀에서 Red, Green, Blue 값을 각각 1 바이트씩 추출하여 별도의 메모리 주소에 분리 저장한다. Red 는 0x41000000, Green 은 0x41002580, Blue 는 0x41004B00 부터 시작하는 메모리 공간에 저장되며, Alpha 는 연산에 사용되지 않기 때문에 건너뛰어진다. 이 과정을 9,600 번 반복하여 각 색상 채널이 독립된 형태로 메모리에 저장된다.

다음으로 `re_rgb_negative` 함수는 앞서 분리 저장된 R, G, B 데이터를 각각 읽어와 색 반전 처리를 수행한다. 색 반전은 255에서 해당 색상 값을 빼는 방식으로 이루어진다. 예를 들어 R 값이 100이라면, 반전된 R 값은 155가 된다. 반전된 R, G, B 값은 다시 연속된 3 바이트로 구성된 RGB 픽셀 형태로 재조합되어 0x20000000부터 시작하는 출력 메모리 영역에 저장된다. 이 과정 또한 9,600 번 반복되며, 최종적으로 24 비트 색상 정보가 저장된 이미지가 생성된다.

이 구조는 원본 RGBA 데이터를 직접 가공하지 않고, 색상 정보를 분리한 뒤 각각의 채널에 대해 독립적인 연산을 수행함으로써 향후 다른 색상 변환 연산이나 병렬 처리를 쉽게 적용할 수 있도록 설계되어 있다. 또한 각 함수는 명확하게 역할이 분리되어 있어 코드의 유지보수성과 확장성이 높다. 비록 단순한 색 반전 연산만을 수행하는 상황에서는 메모리 접근이 증가해 성능상 불리할 수 있으나, 여러 후속 처리가 필요한 고급 영상 처리 응용에서는 유리한 구조이다.



원래 방식은 RGBA 이미지 메모리를 연속적으로 접근하면서 R->G->B->A 를 한 번에 처리하는 (포인터 하나, 순차 접근) 방식이었다. Relocation 방식은 한 번 RGBA 를 읽고, R/G/B 를 따로 저장하는 (Memory copy) 방식이다. 그 후 R, G, B 영역을 각각 독립적으로 접근하여 후처리 하였다.

Register	Value
Current	
R0	0x41002580
R1	0x41004800
R2	0x41007080
R3	0x20007080
R4	0x00000000
R5	0x0000000F
R6	0x0000004B
R7	0x000000ED
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000084
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000084
Mode	Supervisor
States	403223
Sec	0,00672038

Address:	0x20000000
0x20000000:	87 A1 A3 88 8C DC 6D 13 E4 DE F7 4D 88 70 A7 98 10 4A 00 84 20 0C C3 4C 22 62
0x2000001A:	92 7D FD BF 98 C0 08 7F 4C E3 56 D5 6D CD B0 61 3D 3C 3C 06 0F 0E 4A 82 E8 CC
0x20000034:	3A 2D 2E 49 92 14 CA 79 76 22 2F 23 B0 2E B2 CB A2 32 45 3A 93 4E 49 0F 4A 92
0x2000004E:	8B CE 39 D3 2E 38 32 CD 38 51 4E 49 4B 32 E7 24 B9 4B 77 34 EB 49 CE CB 4B 43
0x20000068:	B8 39 2B 93 52 B4 3B CE 0B 44 25 AA 10 BB E8 A2 B9 CB 01 10 E8 B2 30 74 92 C1
0x20000082:	85 1A B2 EC 14 F9 80 92 D9 45 99 86 1A 68 27 5D 0A 0F 04 FC 4E 39 29 CE B2 AC
0x2000009C:	F2 60 2F 4C 0F 34 33 45 20 A0 76 86 5C 96 65 18 A6 80 D4 C0 94 79 B9 08 32 E7
0x200000B6:	02 FA 93 CE 21 B2 1C 47 D1 A5 4E C0 BB 8C 33 A4 23 B9 93 BB C8 C8 92 EE B4 2E
0x200000D0:	D0 A4 32 ED 94 BA 2C 09 93 72 C1 84 1A 18 43 3B 27 4E D3 53 EC 3C CF 29 B1 2E
0x200000EA:	62 90 80 5D 1C 85 4C 61 EC 2C E7 CE 92 33 0C FC 94 44 A5 E0 29 CA 65 04 05 89
0x20000104:	74 A3 4A 11 20 15 20 28 0F B7 AA 10 2A B2 93 92 95 BA 2C 05 53 9A 8B DC 14 38
0x2000011E:	2F B1 39 19 18 96 4E 9C 78 2F 91 C2 A7 06 2A 4C 27 77 5E 92 24 4E 61 D3 28 D6
0x20000138:	51 74 92 0B 85 C2 39 08 05 46 D0 14 0A 18 82 90 66 06 A9 04 90 69 A7 9C 79 1A
0x20000152:	1F 5C F0 C1 84 89 0C 1E 69 87 41 38 01 0D 75 92 CE F9 82 60 34 61 4C 09 04 08
0x2000016C:	86 61 18 1A 94 82 41 05 05 41 74 0A 0A 4D 14 B4 30 30 BC A2 20 1C C2 C9 BD A1
0x20000186:	45 4D 78 48 32 80 A3 1C E9 52 4E 4A 97 5D 72 66 5D 1C D3 C3 5C 08 34 C2 66 92
0x200001A0:	44 8A 0E F8 05 4E 08 40 4A 59 78 49 14 1A FE 93 F9 92 77 71 C9 8D 4E 91 93 D2
0x200001BA:	DC C8 B9 2C 71 3B 5B 8A 78 77 E7 C7 E3 88 C0 CB 0E 0F EB B0 F0 4E C0 3A 9E C3
0x200001D4:	01 9E E8 44 CE E1 04 E7 B1 C7 13 F0 E5 61 1D C0 3A 16 75 8E C7 2F 72 25 04 09
0x200001EE:	D6 1E E5 A2 94 00 8A AE 84 90 6E 84 76 04 CA B0 CA 5D A6 A6 30 EE CA 49 67 9E
0x20000208:	06 A4 5A 2E 46 50 AC B3 E5 68 96 30 28 C9 18 C1 61 77 1A 45 43 0E 63 27 69 D1
0x20000222:	0C 2A 08 C8 8E 21 0E 8D 86 67 38 67 22 7A 0C 38 8E 80 58 38 38 68 E0 81 F0 B0
0x2000023C:	F0 E3 44 87 73 EE A2 2F 8A 2E BC B3 D4 34 C1 A4 A2 50 85 C3 4C 49 5C 76 DE 23
0x20000256:	44 94 07 86 19 15 5C 74 87 2F 41 C4 10 B0 BC 3C 46 22 27 9D 18 37 A6 86 E0 13
0x20000270:	09 24 55 A2 19 BA 95 72 D1 99 6A 52 25 DE 5D 5C E4 05 49 C9 65 72 51 20 50 49

레지스터에 저장된 주소 값과 메모리 0x20000000 에 저장된 결과를 통해 3 과 동일한 결과를 얻었음을 알 수 있다.

Subproject 3 (작성자: 김선영)

- Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      relocate
9          LDR    x0, =0x40000036
10         LDR    x1, =0x41000000
11         LDR    x2, =0x41002580
12         LDR    x3, =0x41004B00
13         MOV    x4, #9600
14
15      loop_relocate
16         LDRB    x5, [x0], #1
17         STRB    x5, [x1], #1
18
19         LDRB    x5, [x0], #1
20         STRB    x5, [x2], #1
21
22         LDRB    x5, [x0], #1
23         STRB    x5, [x3], #1
24
25         ADD    x0, x0, #1      : A 루시
26         SUBS    x4, x4, #1
27         BNE    loop_relocate
28         BX     lr
29
30      re_grayscale
31         LDR    x0, =0x41000000      : R
32         LDR    x1, =0x41002580      : G
33         LDR    x2, =0x41004B00      : B
34         LDR    x3, =0x20000000      : Grayscale 저장 주소
35         MOV    x4, #9600            : 픽셀 수
36
37      loop_gray
38         LDRB    x5, [x0], #1      : R
39         LDRB    x6, [x1], #1      : G
40         LDRB    x7, [x2], #1      : B
41
42         : grayscale = (2R + 6G + B) >> 2
43         ADD    x8, x5, x6, LSL #1
44         ADD    x8, x8, x8, LSL #1
45         ADD    x8, x8, x7
46
47         STRH    x8, [x3], #2      : 1바이트로 저장
48
49         SUBS    x4, x4, #1
50         BNE    loop_gray
51
52      END
```

- 결과

Address	Register	Value
0x40000000: 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 00 00 03 C0	Current	
0x40000001: 00 00 02 80 08 06 00 00 00 53 F9 11 43 00 00 01 73 52 47	R0	0x40000038
0x40000002: 42 00 AE CE 1C E9 00 00 20 00 49 44 54 78 5E 5C 8D 77 73	R1	0x41000001
0x40000003: 23 59 92 EC 1B 29 21 08 B2 44 77 8F 55 71 47 EF B5 77 FF 7B	R2	0x41000039
0x40000004: DF FF F3 3C B3 B5 D0 5D 4D 59 82 02 40 AA 47 3F F7 08 80 B3	R3	0x41004B00
0x40000005: 1C E9 A5 2A 92 00 32 4F 9E 13 C2 C3 A3 F9 F0 F1 B8 B5 7D	R4	0x00002580
0x40000006: 17 73 33 C5 02 75 D1 B6 4D 4C E8 35 94 B3 89 D0 D0 C7 DC 4F	R5	0x0000005E
0x40000007: D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 4D 5B 74 31	R6	0x00000000
0x40000008: C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B D8 46	R7	0x00000000
0x40000009: B4 E7 88 CB 14 D1 B6 31 34 4D B4 BC 47 D7 C6 D4 4C 31 AD 4B	R8	0x00000000
0x4000000A: C4 14 31 F4 B8 D8 DA 55 EF D7 44 17 5D D3 44 34 FE 37 EF 17	R9	0x00000000
0x4000000B: 4D A3 CF B8 4D 0B 3E 7A E5 9F 4D 13 E5 BA 56 7F 6D D8 26 BA	R10	0x00000000
0x4000000C: 66 88 79 E5 97 5A D8 A2 F5 F0 F8 03 BF B1 C6 D6 74 31 4D	R11	0x00000000
0x4000000D: 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 59 79 5A A3 69	R12	0x00000000
0x4000000E: 5A 90 E7 59 7F AE 1B 3F 4B 22 94 46 77 14 CD 18 7D CA 05 4C	R13 (SP)	0x00000000
0x4000000F: 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46	R14 (LR)	0x00000004
0x40000010: 6C 7D 44 37 37 D1 4D 11 4B 13 D1 2F 5B CC CD 12 4B B7 45 D3	R15 (PC)	0x00000038
0x40000011: F6 B1 4C 4D 3E 87 7B E5 E7 D8 BC C4 D8 36 B1 2C AC 45 13 C3	CPSR	0x00000003
0x40000012: 30 C4 D4 4E D1 4C 9D 4F 7F DA A2 E3 7A 2F B3 9E 13 5F D3 18	SPSR	0x00000000
0x40000013: 31 0E 4D CC F3 12 03 6B B8 B6 5A 1F D6 9A 35 9A F8 26 FA 76		
0x40000014: 8B 66 5C 55 EE 5B DF EA DF 5D D7 F0 49 F4 55 EF D5 0F 4D 4C		
0x40000015: 4D C3 6A 45 D3 2C FA AC 65 19 74 23 E8 36 C7 D0 4E D1 C6 E6		
0x40000016: E7 B2 49 B1 43 4D 87 D0 4E 6D 3D 58 F9 B6 D8 B3 D8 96 B8 A1		

Address: 0x20000000		0x2002570	
0x20000000: 6B 12 00 46 01 01 04 7C	0x200257F4: 81 92 95 49		
0x20000008: 21 3A 57 74 24 5C 8C 5A	0x200257F8: 2E 5F 8E 94		
0x20000010: 22 7B 64 BB EF C8 6D 75	0x200257FC: B3 1C 4A 6A		
0x20000018: 7F 83 49 6A CE D3 45 A3	0x20025800: 00 00 00 00		
0x20000020: AB 9D D1 A1 74 A1 A5 67	0x20025804: 00 00 00 00		
0x20000028: 83 71 C0 89 79 B7 72 74	0x20025808: 00 00 00 00		
0x20000030: 8F 6F 6C CA 99 8C 9A 8E	0x2002580C: 00 00 00 00		
0x20000038: 3C 99 78 80 78 B7 F2 61	0x20025810: 00 00 00 00		
0x20000040: 83 40 8B 8C 9F 7E BF 6C	0x20025814: 00 00 00 00		

relocate 함수는 원본 데이터가 저장된 주소에서 4바이트씩 읽어 첫 세 바이트를 각각 세 개의 다른 메모리 영역에 복사하고, 네 번째 바이트는 건너뛴다. 이 작업을 9600번 반복하여 데이터를 세 영역으로 분산해 재배치한다. re_grayscale 함수는 세 개의 메모리 영역에서 각각 R, G, B 값을 1바이트씩 읽어와 그레이스케일 값을 계산한다. 그레이스케일 값은 $(3 \times R + 6 \times G + B)$ 를 8로 나눈 값으로 산출하며, 계산된 값을 16비트 단위로 지정된 메모리 주소에 저장한다. 이 과정을 9600번 반복하여 RGB 데이터를 그레이스케일 이미지로 변환한다. 메모리 0x20000000 주소를 확인하여 3번에서 수행한 코드와 동일한 결과를 얻음을 확인했다.

5. 최적의 ARM 코드 탐색 (작성자: 박대호)

(1) 최적의 ARM 코드를 탐색에 앞서 최적화의 척도를 설정하고자 한다. ARM코드 최적화에는 실행 속도 향상, 코드 크기 최소화, 전력 소비 감소, 캐시 효율 향상, 병렬 처리 활용 등등 여러가지 목표들이 있지만 지금 최적화하려는 코드는 플래너 방식으로 이미지 hex 파일이 재배치된 상태에서 이미지를 변환을 시뮬레이션 하는 것 이므로 명령어 수를 줄여서 코드 크기를 최소화하고 실행 속도를 빠르게 하는 것을 목표로 한다.

(2) 최적화 전략

먼저 명령어의 개수를 줄이기 위해서 하나의 명령어가 여러 동작을 수행하는 복합 명령어들을 적극적으로 사용하기로 했다. 예를 들어 루프를 탈출할 때 NZCV 플래그를 세팅하기 위해 덧셈이나 뺄셈 연산을 수행하고 CMP 명령어를 수행하기 보다는 SUBS나 ADDS 명령어를 사용하기로 했다.

다음으로는 메모리에 접근하는 명령어의 사용을 최소화하기로 했다. 메모리에 접근하는 명령어들은 수행 시간이 길기 때문이다.

마지막으로 루프 언롤링 방식을 활용하여 수행 시간을 줄이기로 했다. 루프 언롤링은 반복문에서 한 번에 처리하는 작업 수를 늘려서, 루프 반복 횟수를 줄이고 분기 명령어 실행 횟수를 줄이는 방법이다. 실험에서 수행한 코드들이 모두 루프를 포함하고 있고 루프 안에서 단순한 연산을 수행하므로 루프 언롤링으로 인한 큰 수행시간 단축이 기대된다.

(3) Function#1- Red pixel count

```

1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL Red_Pixel_Count_Re
7
8      relocate
9          LDR    r0, =0x40000036 ; r0 = RGBA 데이터 시작 주소
10         LDR    r1, =0x41000000 ; r1 = R값 저장할 목적지 주소
11         LDR    r2, =0x41002580 ; r2 = G값 저장할 목적지 주소
12         LDR    r3, =0x41004B00 ; r3 = B값 저장할 목적지 주소
13         MOV    r4, #9600 ; r4 = 픽셀 수
14
15      loop_relocate
16         LDRB   r5, [r0], #1 ;
17         STRB   r5, [r1], #1 ; R 저장
18
19         LDRB   r5, [r0], #1 ;
20         STRB   r5, [r2], #1 ; G 저장
21
22         LDRB   r5, [r0], #1 ;
23         STRB   r5, [r3], #1 ; B 저장
24
25         ADD    r0, r0, #1 ; A 무시
26
27         SUBS   r4, r4, #1
28
29         BNE    loop_relocate
30         BX     lr ; 재정의 끝
31
32      Red_Pixel_Count_Re
33
34         LDR    r0, =0x41000000
35         MOV    r1, #9600 ; r1: 총 픽셀 수_9600개
36         MOV    r2, #0 ; r2: 조건 만족하는 픽셀 수 count
37         MOV    r3, #0 ; r3: 현재 인덱스
38
39      loop
40         CMP    r3, r1
41         BEQ    done
42
43         LDRB   r4, [r0]
44         CMP    r4, #128
45         BLT    skip
46         ADD    r2, r2, #1
47
48      skip
49         ADD    r0, r0, #1
50         ADD    r3, r3, #1
51         B      loop
52
53      done
54         MOV    r0, r2
55         BX     lr
56
57      END
58

```

<기존 코드>

```

1      AREA    RelocateAndCount, CODE, READONLY
2      EXPORT  relocate_and_count_red
3      ENTRY
4
5      relocate_and_count_red
6          PUSH    {R4-R9, LR}
7
8          MOV     R1, #0
9          LDR     R2, =0x40000036          ; RGBA ?? ??
10         LDR     R3, =0x50000000          ; R ???
11         LDR     R4, =0x50002580          ; G ???
12         LDR     R5, =0x50004B00          ; B ???
13
14         ; === [1??] RGBA -> RRR... GGG... BBB... ?? ?? ===
15         relocate_loop
16             CMP     R1, #9600
17             BGE     count_start
18
19             LDRB    R6, [R2]              ; R
20             LDRB    R7, [R2, #1]          ; G
21             LDRB    R8, [R2, #2]          ; B
22
23             STRB    R6, [R3], #1          ; R ??
24             STRB    R7, [R4], #1          ; G ??
25             STRB    R8, [R5], #1          ; B ??
26
27             ADD     R2, R2, #4
28             ADD     R1, R1, #1
29             B       relocate_loop
30
31 count_start
32     MOV     R0, #0                      ; ??? ???
33     MOV     R1, #1600                   ; ?? ??
34     LDR     R3, =0x50000000             ; R ?? ?? ??
35
36 count_loop
37     LDRB    R6, [R3], #1                ; ? ?? ??? ??
38     CMP     R6, #128                    ;
39     ADDGE   R0, R0, #1                  ; ?? ?? ? ??? ??
40
41     LDRB    R6, [R3], #1                ; ? ?? ??? ??
42     CMP     R6, #128                    ;
43     ADDGE   R0, R0, #1                  ;
44
45     LDRB    R6, [R3], #1                ; ? ?? ??? ??
46     CMP     R6, #128                    ;
47     ADDGE   R0, R0, #1                  ;
48
49     LDRB    R6, [R3], #1                ; ? ?? ??? ??
50     CMP     R6, #128                    ;
51     ADDGE   R0, R0, #1                  ;
52
53     LDRB    R6, [R3], #1                ; ? ?? ??? ??
54     CMP     R6, #128                    ;
55     ADDGE   R0, R0, #1                  ;
56
57     LDRB    R6, [R3], #1                ; ? ?? ??? ??
58     CMP     R6, #128                    ;
59     ADDGE   R0, R0, #1                  ;
60
61
62     SUBS    R1, R1, #1                  ; ??? ?? ? ?? ??? ???
63     BNE     count_loop
64
65 done
66     POP     {R4-R9, PC}
67
68     END
69
70

```

<최적화 후 코드>

변경사항: function#1 의 경우 메모리에 이미지 파일을 재배치한 후 레드 픽셀의 개수를 센 코드와 그냥 4 바이트씩 접근하여 픽셀을 센 코드의 속도 차이가 없었기 때문에 코드를 새로 구성했다. 기능적으로는 차이가 없다. 먼저 relocate_loop 까지 실행해서 0x50000000 에 색상 채널들을 재배치하고 128 비트 이상의 레드 픽셀 수를 1 바이트씩 접근하여 계산한다.

128 비트 레드 픽셀을 세는 과정에서 우선 명령어 수를 줄이기 위해 ADDGE 를 활용했다. 최적화 이전 코드에서는 이 과정을 skip 하는 함수를 새로 만들어 분기까지 해야 했지만 최적화 코드에서는 ADDGE 를 통해 한 번에 수행하였다. 그리고 나서는 수행 시간을 줄이기 위해서 루프 언롤링을 수행했다. 4,6,8 씩 루프 언롤링 횟수를 늘려가면서 실험했는데 루프 언롤링 횟수를 늘릴수록 시간이 줄긴 했지만 유지보수와 코드의 가독성 등을 고려하여 언롤링 횟수는 6 으로 하였다.

Register	Value
Current	
R0	0x000012C3
R1	0x00000000
R2	0x40009636
R3	0x50002580
R4	0x50004B00
R5	0x50007080
R6	0x00000020
R7	0x000000B4
R8	0x00000012
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0xFFFFFE4
R14 (LR)	0x00000000
R15 (PC)	0x000000A0
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x000000A0
Mode	Supervisor
States	265622
Sec	0.00442703

<최적화 코드 전체 수행시간>

Register	Value
Current	
R0	0x00000000
R1	0x00002580
R2	0x40009636
R3	0x50002580
R4	0x50004B00
R5	0x50007080
R6	0x00000020
R7	0x000000B4
R8	0x00000012
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0xFFFFFE4
R14 (LR)	0x00000000
R15 (PC)	0x00000044
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000044
Mode	Supervisor
States	211221
Sec	0.00352035

<최적화 코드 메모리 재배치 시간>

전체 수행시간 - 메모리 재배치 시간 = 이미지 변환 함수 수행시간 =
0.00090668sec

R0 값을 통해 코드가 이전과 같은 결과를 얻었음을 알 수 있다.

수행시간 단축은 기존의 약 2.1ms 보다 1.2ms 줄어든 수행시간을 얻었다. 레드 픽셀의 수를 세는 루프에서는 기존에 픽셀당 명령어 수가 기준(128 비트 이상)을 만족 시 8 개 불만족 시 9 개였다면 최적화 코드에서는 모든 픽셀이 픽셀당 5 개이다. 128 비트 이상은 레드 픽셀 수가 약 50% 정도였으므로 픽셀당 명령어 수를 4.5 개 단축했다.

(4) Function#2-Image negative

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #9600
36
37     loop_negative
38         LDRB    r5, [r0], #1
39         RSBS    r5, r5, #255
40         STRB    r5, [r3], #1
41
42         LDRB    r6, [r1], #1
43         RSBS    r6, r6, #255
44         STRB    r6, [r3], #1
45
46         LDRB    r7, [r2], #1
47         RSBS    r7, r7, #255
48         STRB    r7, [r3], #1
49
50         SUBS    r4, r4, #1
51         BNE    loop_negative
52
53         BX     lr
54
55     END
```

<기존코드>

```

1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #1600
36
37     loop_negative
38         LDRB    r5, [r0], #1
39         RSB     r5, r5, #255
40
41
42         LDRB    r6, [r1], #1
43         RSB     r6, r6, #255
44
45
46         LDRB    r7, [r2], #1
47         RSB     r7, r7, #255
48
49
50         STRB    r5, [r3], #1
51         STRB    r6, [r3], #1
52         STRB    r7, [r3], #1
53

```

```
54      LDRB    r5, [r0], #1
55      RSB     r5, r5, #255
56
57
58      LDRB    r6, [r1], #1
59      RSB     r6, r6, #255
60
61
62      LDRB    r7, [r2], #1
63      RSB     r7, r7, #255
64
65
66      STRB    r5, [r3], #1
67      STRB    r6, [r3], #1
68      STRB    r7, [r3], #1
69
70      LDRB    r5, [r0], #1
71      RSB     r5, r5, #255
72
73
74      LDRB    r6, [r1], #1
75      RSB     r6, r6, #255
76
77
78      LDRB    r7, [r2], #1
79      RSB     r7, r7, #255
80
81
82      STRB    r5, [r3], #1
83      STRB    r6, [r3], #1
84      STRB    r7, [r3], #1
85
```

```

86         LDRB     r5, [r0], #1
87         RSB      r5, r5, #255
88
89
90         LDRB     r6, [r1], #1
91         RSB      r6, r6, #255
92
93
94         LDRB     r7, [r2], #1
95         RSB      r7, r7, #255
96
97
98         STRB     r5, [r3], #1
99         STRB     r6, [r3], #1
100        STRB     r7, [r3], #1
101
102        LDRB     r5, [r0], #1
103        RSB      r5, r5, #255
104
105
106        LDRB     r6, [r1], #1
107        RSB      r6, r6, #255
108
109
110        LDRB     r7, [r2], #1
111        RSB      r7, r7, #255
112
113
114        STRB     r5, [r3], #1
115        STRB     r6, [r3], #1
116        STRB     r7, [r3], #1
117
118        LDRB     r5, [r0], #1
119        RSB      r5, r5, #255
120
121
122        LDRB     r6, [r1], #1
123        RSB      r6, r6, #255
124
125
126        LDRB     r7, [r2], #1
127        RSB      r7, r7, #255
128
129
130        STRB     r5, [r3], #1
131        STRB     r6, [r3], #1
132        STRB     r7, [r3], #1
133
134        SUBS     r4, r4, #1
135        BNE      loop_negative
136
137        BX      lr
138
139        END

```

<최적화 후 코드>

Address:	0x20000000	
0x20000000:	87 A1 A3 88 8C DC 6D 13 E4 DE F7 4D 88 70 A7 98 10 4A 00 84 20 0C C3 4C 22 62 92 7D FD BF	
0x20000001E:	98 C0 08 7F 4C E3 56 D5 6D CD B0 61 3D 3C 3C 06 0F 0E 4A 82 E8 CC 3A 2D 2E 49 92 14 CA 79	
0x20000003C:	76 22 2F 23 B0 2E B2 CB A2 32 45 3A 93 4E 49 0F 4A 92 8B CE 39 D3 2E 38 32 CD 38 51 4E 49	
0x20000005A:	4B 32 E7 24 B9 4B 77 34 EB 49 CE CB 4B 43 B8 39 2B 93 52 B4 3B CE 0B 44 25 AA 10 BB E8 A2	
0x200000078:	B9 CB 01 10 E8 B2 30 74 92 C1 85 1A B2 EC 14 F9 80 92 D9 45 99 86 1A 68 27 5D 0A 0F 04 FC	
0x200000096:	4E 39 29 CE B2 AC F2 60 2F 4C 0F 34 33 45 20 A0 76 86 5C 96 65 18 A6 80 D4 C0 94 79 B9 08	
0x2000000B4:	32 E7 02 FA 93 CE 21 B2 1C 47 D1 A5 4E C0 BB 8C 33 A4 23 B9 93 BB C8 C8 92 EE B4 2E D0 A4	
0x2000000D2:	32 ED 94 BA 2C 09 93 72 C1 84 1A 18 43 3B 27 4E D3 53 EC 3C CF 29 B1 2E 62 90 80 5D 1C 85	
0x2000000F0:	4C 61 EC 2C E7 CE 92 33 0C FC 94 44 A5 E0 29 CA 65 04 05 89 74 A3 4A 11 20 15 20 28 0F B7	
0x20000010E:	AA 10 2A B2 93 92 95 BA 2C 05 53 9A 8B DC 14 38 2F B1 39 19 18 96 4E 9C 78 2F 91 C2 A7 06	
0x20000012C:	2A 4C 27 77 5E 92 24 4E 61 D3 28 D6 51 74 92 0B 85 C2 39 08 05 46 D0 14 0A 18 82 90 66 06	
0x20000014A:	A9 04 90 69 A7 9C 79 1A 1F 5C F0 C1 84 89 0C 1E 69 87 41 38 01 0D 75 92 CE F9 82 60 34 61	
0x200000168:	4C 09 04 08 86 61 18 1A 94 82 41 05 05 41 74 0A 0A 4D 14 B4 30 30 BC A2 20 1C C2 C9 BD A1	
0x200000186:	45 4D 78 48 32 80 A3 1C E9 52 4E 4A 97 5D 72 66 5D 1C D3 C3 5C 08 34 C2 66 92 44 8A 0E F8	
0x2000001A4:	05 4E 08 40 4A 59 78 49 14 1A FE 93 F9 92 77 71 C9 8D 4E 91 93 D2 DC C8 B9 2C 71 3B 5B 8A	
0x2000001C2:	78 77 E7 C7 E3 88 C0 CB 0E 0F EB B0 F0 4E C0 3A 9E C3 01 9E E8 44 CE E1 04 E7 B1 C7 13 F0	
0x2000001E0:	E5 61 1D C0 3A 16 75 8E C7 2F 72 25 04 09 D6 1E E5 A2 94 00 8A AE 84 90 6E 84 76 04 CA B0	
0x2000001FE:	CA 5D A6 A6 30 EE CA 49 67 9E 06 A4 5A 2E 46 50 AC B3 E5 68 96 30 28 C9 18 C1 61 77 1A 45	
0x20000021C:	43 0E 63 27 69 D1 0C 2A 08 C8 8E 21 0E 8D 86 67 38 67 22 7A 0C 38 8E 80 58 38 38 68 E0 81	
0x20000023A:	F0 B0 F0 E3 44 87 73 EE A2 2F 8A 2E BC B3 D4 34 C1 A4 A2 50 85 C3 4C 49 5C 76 DE 23 44 94	
0x200000258:	07 86 19 15 5C 74 87 2F 41 C4 10 B0 BC 3C 46 22 27 9D 18 37 A6 86 E0 13 09 24 55 A2 19 BA	
0x200000276:	95 72 D1 99 6A 52 25 DE 5D 5C E4 05 49 C9 65 72 51 20 50 49 96 29 2F 41 8C 08 16 D2 08 70	
0x200000294:	5A 13 F2 C0 16 C0 8A 12 BC 81 2D 82 46 10 06 42 09 81 26 80 21 85 02 65 D0 29 2F 19 73 51	
0x2000002B2:	53 D4 80 A1 18 67 34 4E F1 60 D3 49 69 29 FB 48 26 A9 52 0C 93 A4 CB 63 39 84 60 AF 89 98	
0x2000002D0:	29 67 46 11 CC 13 0F E6 18 41 29 03 E0 4E 52 8B 42 30 69 40 96 A7 A7 48 B2 71 72 61 86 F0	
0x2000002FF:	4B 83 2F F4 69 9C 5D 66 4F 13 7F B3 82 80 69 67 29 94 14 B4 51 F3 64 09 28 3A 46 8B 32 B2	

Registers	
Register	Value
Current	
R0	0x41002580
R1	0x41004800
R2	0x41007080
R3	0x20007080
R4	0x00000000
R5	0x000000DF
R6	0x0000004B
R7	0x000000ED
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000138
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000138
Mode	Supervisor
States	371223
Sec	0.00618705

<전체 수행시간>

Register	Value
Current	
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000044
CPSR	0x600000D3
SPSR	0x00000000
+	User/System
+	Fast Interrupt
+	Interrupt
+	Supervisor
+	Abort
+	Undefined
-	Internal
PC \$	0x00000044
Mode	Supervisor
States	192016
Sec	0.00320027

<메모리 재배치 수행시간>

전체 수행시간 - 메모리 재배치 수행시간 = 이미지 변환 수행시간 =
0.00298678sec

변경사항: 0x20000000 메모리 주소에 저장된 값을 통해 최적화 후 코드가 이전과 같은 결과를 갖는다는 사실을 확인했다.

Function#2에서는 이미지를 negative 를 변환하는 코드가 SUBS 를 사용하는 등 최적화가 잘 되어 있어서 명령어 수를 따로 줄이지는 않았다. 하지만 RSBS 가 사용될 이유가 없었기 때문에 RSB 를 사용했고 레지스터 연산을 먼저하고 메모리에 쓰기를 나중에 하기 위해 코드 순서를 변경하였다. Function#1 에서와 동일한 이유로 루프 언롤링은 6 번으로 실행하였다. 이미지 변환 함수 수행시간은 약 3.5ms 에서 약 2.98ms 로 0.5ms 정도 단축됐다. 명령어 개수의 감소는 없다.

(5) Function#3-Grayscale

```

1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      relocate
9          LDR     x0, =0x40000036
10         LDR     x1, =0x41000000
11         LDR     x2, =0x41002580
12         LDR     x3, =0x41004B00
13         MOV     x4, #9600
14
15      loop_relocate
16         LDRB     x5, [x0], #1
17         STAB     x5, [x1], #1
18
19         LDRB     x5, [x0], #1
20         STAB     x5, [x2], #1
21
22         LDRB     x5, [x0], #1
23         STAB     x5, [x3], #1
24
25         ADD     x0, x0, #1      ; A 루프
26         SUBS     x4, x4, #1
27         BNE     loop_relocate
28         BX      lr
29
30      re_grayscale
31         LDR     x0, =0x41000000      ; R
32         LDR     x1, =0x41002580      ; G
33         LDR     x2, =0x41004B00      ; B
34         LDR     x3, =0x20000000      ; Grayscale 저장 주소
35         MOV     x4, #9600            ; 픽셀 수
36
37      loop_gray
38         LDRB     x5, [x0], #1      ; R
39         LDRB     x6, [x1], #1      ; G
40         LDRB     x7, [x2], #1      ; B
41
42         ; grayscale = (2R + 6G + B) >> 2
43         ADD     x8, x5, x6, LSL #1
44         ADD     x8, x8, x6, LSL #1
45         ADD     x8, x8, x7
46
47         STRH     x8, [x3], #2      ; 1바이트로 저장
48
49         SUBS     x4, x4, #1
50         BNE     loop_gray
51
52      END

```

<기존 코드>

```

1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      loop_end
9          B loop_end          ; ?? ??
10
11     relocate
12         LDR    r0, =0x4000003e
13         LDR    r1, =0x41000000
14         LDR    r2, =0x41002580
15         LDR    r3, =0x41004B00
16         MOV    r4, #9600
17
18     loop_relocate
19         LDRB   r5, [r0], #1
20         STRB   r5, [r1], #1
21
22         LDRB   r5, [r0], #1
23         STRB   r5, [r2], #1
24
25         LDRB   r5, [r0], #1
26         STRB   r5, [r3], #1
27
28         ADD    r0, r0, #1          ; A ??
29         SUBS   r4, r4, #1
30         BNE    loop_relocate
31         BX     lr
32
33     re_grayscale
34         LDR    r0, =0x41000000      ; R
35         LDR    r1, =0x41002580      ; G
36         LDR    r2, =0x41004B00      ; B
37         LDR    r3, =0x20000000      ; Grayscale ?? ??
38         MOV    r4, #1600            ; ?? ?
39
40     loop_gray
41         LDRB   r5, [r0], #1          ; R
42         LDRB   r6, [r1], #1          ; G
43         LDRB   r7, [r2], #1          ; B
44
45         ; grayscale = (3R + 6G + B) >> 3
46         ADD    r8, r5, r6, LSL #1
47         ADD    r8, r8, r8, LSL #1
48         ADD    r8, r8, r7
49
50         STRH   r8, [r3], #2          ; 1???? ??
51
52         LDRB   r5, [r0], #1          ; R
53         LDRB   r6, [r1], #1          ; G
54         LDRB   r7, [r2], #1          ; B
55
56         ; grayscale = (3R + 6G + B) >> 3
57         ADD    r8, r5, r6, LSL #1
58         ADD    r8, r8, r8, LSL #1
59         ADD    r8, r8, r7
60
61         STRH   r8, [r3], #2          ; 1???? ??
62

```

```

63     LDRB     r5, [r0], #1      ; R
64     LDRB     r6, [r1], #1      ; G
65     LDRB     r7, [r2], #1      ; B
66
67     ; grayscale = (3R + 6G + B) >> 3
68     ADD      r8, r5, r6, LSL #1
69     ADD      r8, r8, r8, LSL #1
70     ADD      r8, r8, r7
71
72     STRH     r8, [r3], #2      ; 1???? ??
73
74
75
76     LDRB     r5, [r0], #1      ; R
77     LDRB     r6, [r1], #1      ; G
78     LDRB     r7, [r2], #1      ; B
79
80     ; grayscale = (3R + 6G + B) >> 3
81     ADD      r8, r5, r6, LSL #1
82     ADD      r8, r8, r8, LSL #1
83     ADD      r8, r8, r7
84
85     STRH     r8, [r3], #2
86
87     LDRB     r5, [r0], #1      ; R
88     LDRB     r6, [r1], #1      ; G
89     LDRB     r7, [r2], #1      ; B
90
91     ; grayscale = (3R + 6G + B) >> 3
92     ADD      r8, r5, r6, LSL #1
93     ADD      r8, r8, r8, LSL #1
94     ADD      r8, r8, r7
95
96     STRH     r8, [r3], #2
97
98     LDRB     r5, [r0], #1      ; R
99     LDRB     r6, [r1], #1      ; G
100    LDRB     r7, [r2], #1      ; B
101
102    ; grayscale = (3R + 6G + B) >> 3
103    ADD      r8, r5, r6, LSL #1
104    ADD      r8, r8, r8, LSL #1
105    ADD      r8, r8, r7
106
107    STRH     r8, [r3], #2
108
109    SUBS     r4, r4, #1
110    BNE      loop_gray
111
112    B loop_end
113
114    END

```

<최적화 후 코드>

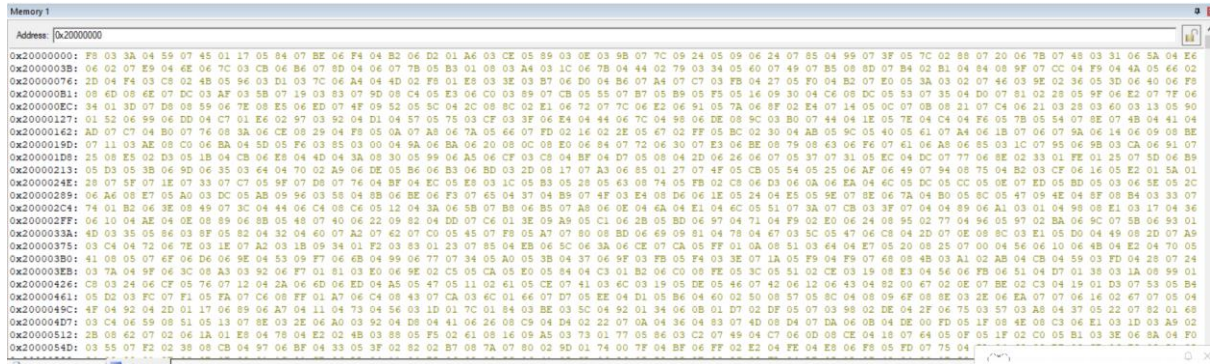
Register	Value
Current	
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20004B00
R4	0x00000000
R5	0x00000020
R6	0x000000B4
R7	0x00000012
R8	0x000004AA
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x0000010C
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x0000010C
Mode	Supervisor
States	332823
Sec	0,00554705

<전체 코드 수행시간>

Register	Value
Current	
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000048
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000048
Mode	Supervisor
States	192016
Sec	0,00320027

<메모리 재배치 수행시간>

전체 수행시간 - 메모리 재배치 수행시간 = grayscale 변환 시간 =
0.00234678sec



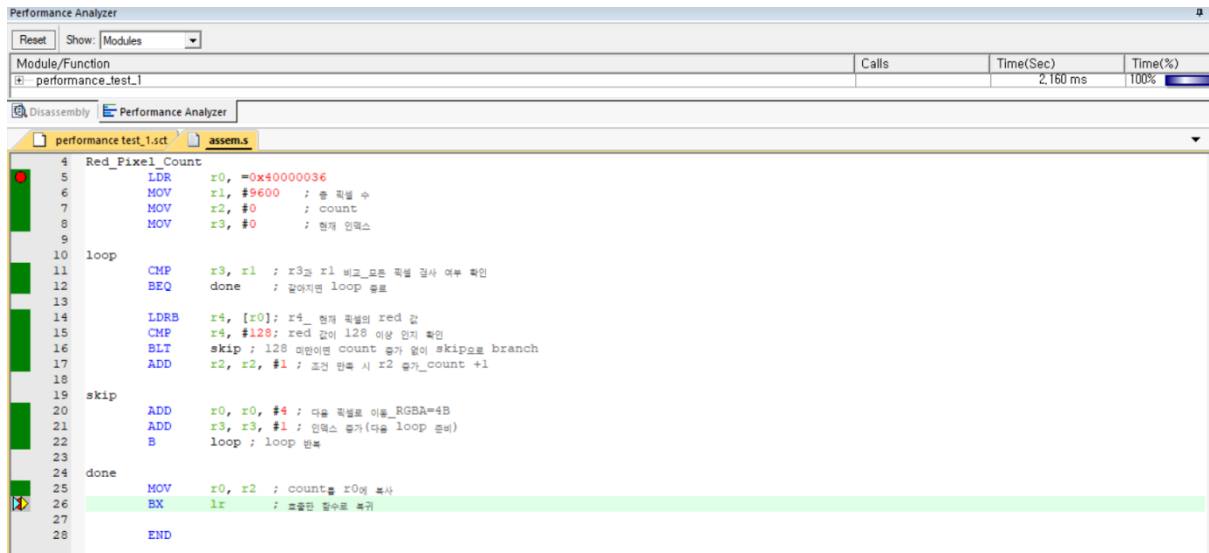
변경사항: 메모리 주소 0x20000000 값을 확인하여 기존의 코드와 동일한 동작을 수행함을 확인했다. Function#3 코드에서도 기존의 코드가 명령어 최적화가 잘 되어 있어서 개수를 줄이지는 않았다. 하지만 기존의 코드의 경우 코드를 끝까지 수행하는 데에는 문제가 없었지만 수행이 완료되면 start 가 b1로 호출된 것이 아니기 때문에 lr에 정상 리턴 주소가 없어 코드를 모두 실행한 후 쓰레기 값으로 점프하는 현상이 있었다. 그래서 loop end를 추가하여 안정화시켰다. 실행 시간을 줄이기 위해서는 위에서와 마찬가지로 루프 언롤링을 6으로 설정했다. 결과적으로 기존에 약 2.9ms의 수행시간에서 약 2.3ms로 약 0.6ms의 수행시간 단축을 달성했다. 이미지 변환 함수의 명령어 개수 단축은 일어나지 않았다.

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

Subproject 1 (작성자: 최은지)

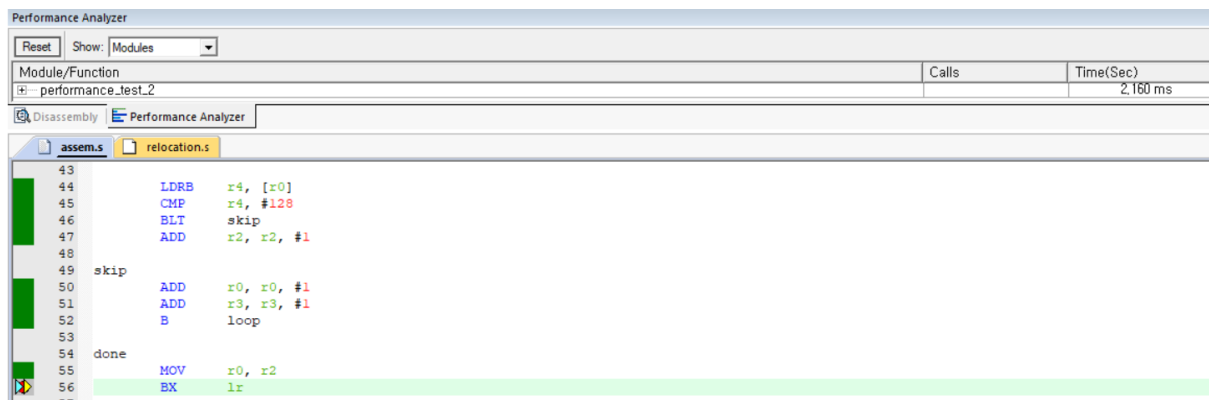
: 성능 비교에 Performance analyzer를 통해 함수 수행 시간 사용

- 메모리 재배열 전



2.160[ms]

- 메모리 재배열 후



count하는 함수만을 생각했을 때 2.160[ms]

기존 구조에서는 RGBA 순서로 메모리에 저장되어 반복문에서 각 색상 값을 4 바이트씩 순차적으로 접근하게 되며, 이로 인해 CPU 캐시의 spatial locality를 제대로 활용하지 못한다. 반면 재배열 이후에는 R, G, B 값을 각각 연속된 공간에 저장함으로써 동일한 유형의 데이터를 한꺼번에 접근할 수 있게 되어 캐시 miss를 줄이고 접근 효율을 높일 수 있다. 또한 불필요한 메모리 접근이 줄어들면서 전체 메모리 접근 횟수도 감소한다. 이처럼 메모리 재배열은 접근

패턴을 규칙적이고 예측 가능하게 해 전반적인 함수 수행 시간을 줄이는 효과를 가져온다. 이론적으로는 성능이 향상되어야 함에도 불구하고, 실제 실행 결과에서는 유의미한 차이가 나타나지 않았다. 이는 감소된 실행 시간이 너무 미미해 측정 상으로 드러나지 않았거나, 코드 구조가 여전히 비효율적이기 때문일 것으로 예측된다. 성능 차이를 명확히 확인하기 위해서는 명령어의 수를 조절하거나 코드 최적화 과정이 필요하다.

Subproject 2 (작성자: 유지원)

[성능 비교표]

항목	원본 방식	리로케이션 방식
루프 수	1 (9600 회)	1(9600 회)
명령어 수 / 픽셀	약 7	약 12~14
메모리 접근 수 / 픽셀	7 (4 read + 3 write)	13 (6 read + 7 write)
총 명령 실행 수	약 67,200	약 124,800 이상
코드 구조	단순	모듈화, 재사용 가능
캐시 효율	높음 (연속 접근)	낮음 (분리 접근)
병렬성 / 확장성	낮음	높음

Internal	
PC \$	0x0000003C
Mode	Supervisor
States	220801
Sec	0,00368002

<메모리 재배치 전 수행시간>

Internal	
PC \$	0x00000084
Mode	Supervisor
States	403223
Sec	0,00672038

Internal	
PC \$	0x00000044
Mode	Supervisor
States	192016
Sec	0,00320027

<메모리 재배치 후 전체 코드 수행시간 & 재배치만 하는데

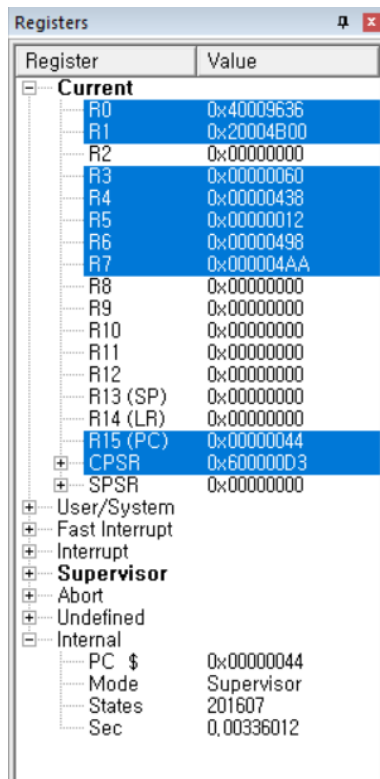
걸린 시간>

전체 수행 시간 - 재배치 시간 = 이미지 변환 함수 수행 시간 = 0.00352011 sec

약 0.2msec 의 수행시간이 감소했다. 메모리에 재배치 후 negative 값으로 변환하는 과정에서 바이트 접근이 1 바이트씩 접근하는 방식으로 바뀌어서 속도가 줄어든 것으로 보인다.

Subproject 3 (작성자: 김선영)

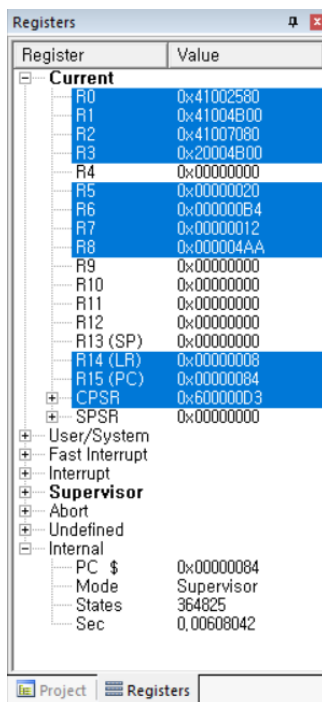
- 메모리 재배열 전



Register	Value
Current	
R0	0x40009636
R1	0x20004B00
R2	0x00000000
R3	0x00000060
R4	0x00000438
R5	0x00000012
R6	0x00000498
R7	0x000004AA
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000044
CPSR	0x60000003
SPSR	0x00000000
Supervisor	
PC \$	0x00000044
Mode	Supervisor
States	201607
Sec	0,00336012

수행시간 = 0.00336012sec

- 메모리 재배열 후



Register	Value
Current	
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20004B00
R4	0x00000000
R5	0x00000020
R6	0x000000B4
R7	0x00000012
R8	0x000004AA
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000084
CPSR	0x60000003
SPSR	0x00000000
Supervisor	
PC \$	0x00000084
Mode	Supervisor
States	364825
Sec	0,00608042

<전체코드 수행시간>

Register	Value
Current	
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000044
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000044
Mode	Supervisor
States	192016
Sec	0,00320027

<메모리 재배치 수행시간>

그레이스케일 변환 시간 = 전체 수행시간 - 메모리 재배치 수행시간 =
0.00288015sec

그레이스케일 계산을 수행하는 어셈블리어 코드를 재배치한 결과, 전체 실행 시간에서 실제 연산이 아닌 단순 재배치에 소요된 시간을 제외한 순수 연산 시간만을 기준으로 성능을 분석하였다. 특히, 이미지 데이터를 처리할 때 이전 방식보다 1 바이트씩 순차적으로 접근하도록 코드를 최적화함으로써, 메모리 접근의 효율성이 향상되어 전체적인 실행 속도가 빨라졌다.

7. 결과물(최종 코드)

Function#1

```
1      AREA    RelocateAndCount, CODE, READONLY
2      EXPORT  relocate_and_count_red
3      ENTRY
4
5      relocate_and_count_red
6          PUSH    {R4-R9, LR}
7
8          MOV     R1, #0
9          LDR     R2, =0x40000036          ; RGBA ?? ??
10         LDR     R3, =0x50000000          ; R ???
11         LDR     R4, =0x50002580          ; G ???
12         LDR     R5, =0x50004B00          ; B ???
13
14         ; === [1??] RGBA -> RRR... GGG... BBB... ?? ?? ===
15         relocate_loop
16             CMP     R1, #9600
17             BGE     count_start
18
19             LDRB    R6, [R2]              ; R
20             LDRB    R7, [R2, #1]          ; G
21             LDRB    R8, [R2, #2]          ; B
22
23             STRB    R6, [R3], #1          ; R ??
24             STRB    R7, [R4], #1          ; G ??
25             STRB    R8, [R5], #1          ; B ??
26
27             ADD     R2, R2, #4
28             ADD     R1, R1, #1
29             B       relocate_loop
```

```

32 count_start
33     MOV     R0, #0                ; ??? ???
34     MOV     R1, #1600            ; ?? ??
35     LDR     R3, =0x50000000      ; R ?? ?? ??
36
37 count_loop
38     LDRB    R6, [R3], #1          ; ? ?? ??? ??
39     CMP     R6, #128              ;
40     ADDGE   R0, R0, #1            ; ?? ?? ? ??? ??
41
42     LDRB    R6, [R3], #1          ; ? ?? ??? ??
43     CMP     R6, #128              ;
44     ADDGE   R0, R0, #1            ;
45
46     LDRB    R6, [R3], #1          ; ? ?? ??? ??
47     CMP     R6, #128              ;
48     ADDGE   R0, R0, #1            ;
49
50     LDRB    R6, [R3], #1          ; ? ?? ??? ??
51     CMP     R6, #128              ;
52     ADDGE   R0, R0, #1            ;
53
54     LDRB    R6, [R3], #1          ; ? ?? ??? ??
55     CMP     R6, #128              ;
56     ADDGE   R0, R0, #1            ;
57
58     LDRB    R6, [R3], #1          ; ? ?? ??? ??
59     CMP     R6, #128              ;
60     ADDGE   R0, R0, #1            ;
61
62
63     SUBS    R1, R1, #1            ; ??? ?? ? ?? ??? ???
64     BNE     count_loop
65
66 done
67     POP     {R4-R9, PC}
68
69     END
70

```

Function#2

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15      loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
30      re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #1600
36
37      loop_negative
38         LDRB    r5, [r0], #1
39         RSB     r5, r5, #255
40
41
42         LDRB    r6, [r1], #1
43         RSB     r6, r6, #255
44
45
46         LDRB    r7, [r2], #1
47         RSB     r7, r7, #255
48
49
50         STRB    r5, [r3], #1
51         STRB    r6, [r3], #1
52         STRB    r7, [r3], #1
53
```

```
54      LDRB    r5, [r0], #1
55      RSB     r5, r5, #255
56
57
58      LDRB    r6, [r1], #1
59      RSB     r6, r6, #255
60
61
62      LDRB    r7, [r2], #1
63      RSB     r7, r7, #255
64
65
66      STRB    r5, [r3], #1
67      STRB    r6, [r3], #1
68      STRB    r7, [r3], #1
69
70      LDRB    r5, [r0], #1
71      RSB     r5, r5, #255
72
73
74      LDRB    r6, [r1], #1
75      RSB     r6, r6, #255
76
77
78      LDRB    r7, [r2], #1
79      RSB     r7, r7, #255
80
81
82      STRB    r5, [r3], #1
83      STRB    r6, [r3], #1
84      STRB    r7, [r3], #1
85
```

```

86         LDRB     r5, [r0], #1
87         RSB      r5, r5, #255
88
89
90         LDRB     r6, [r1], #1
91         RSB      r6, r6, #255
92
93
94         LDRB     r7, [r2], #1
95         RSB      r7, r7, #255
96
97
98         STRB     r5, [r3], #1
99         STRB     r6, [r3], #1
100        STRB     r7, [r3], #1
101
102        LDRB     r5, [r0], #1
103        RSB      r5, r5, #255
104
105
106        LDRB     r6, [r1], #1
107        RSB      r6, r6, #255
108
109
110        LDRB     r7, [r2], #1
111        RSB      r7, r7, #255
112
113
114        STRB     r5, [r3], #1
115        STRB     r6, [r3], #1
116        STRB     r7, [r3], #1
117
118        LDRB     r5, [r0], #1
119        RSB      r5, r5, #255
120
121
122        LDRB     r6, [r1], #1
123        RSB      r6, r6, #255
124
125
126        LDRB     r7, [r2], #1
127        RSB      r7, r7, #255
128
129
130        STRB     r5, [r3], #1
131        STRB     r6, [r3], #1
132        STRB     r7, [r3], #1
133
134        SUBS     r4, r4, #1
135        BNE      loop_negative
136
137        BX       lr
138
139        END

```

Function#3

```

1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      loop_end
9          B loop_end          ; ?? ??
10
11     relocate
12         LDR    r0, =0x40000036
13         LDR    r1, =0x41000000
14         LDR    r2, =0x41002580
15         LDR    r3, =0x41004B00
16         MOV    r4, #9600
17
18     loop_relocate
19         LDRB   r5, [r0], #1
20         STRB   r5, [r1], #1
21
22         LDRB   r5, [r0], #1
23         STRB   r5, [r2], #1
24
25         LDRB   r5, [r0], #1
26         STRB   r5, [r3], #1
27
28         ADD    r0, r0, #1          ; A ??
29         SUBS   r4, r4, #1
30         BNE    loop_relocate
31         BX     lr
32
33     re_grayscale
34         LDR    r0, =0x41000000    ; R
35         LDR    r1, =0x41002580    ; G
36         LDR    r2, =0x41004B00    ; B
37         LDR    r3, =0x20000000    ; Grayscale ?? ??
38         MOV    r4, #1600          ; ?? ?
39
40     loop_gray
41         LDRB   r5, [r0], #1        ; R
42         LDRB   r6, [r1], #1        ; G
43         LDRB   r7, [r2], #1        ; B
44
45         ; grayscale = (3R + 6G + B) >> 3
46         ADD    r8, r5, r6, LSL #1
47         ADD    r8, r8, r8, LSL #1
48         ADD    r8, r8, r7
49
50         STRH   r8, [r3], #2        ; 1???? ??
51
52         LDRB   r5, [r0], #1        ; R
53         LDRB   r6, [r1], #1        ; G
54         LDRB   r7, [r2], #1        ; B
55
56         ; grayscale = (3R + 6G + B) >> 3
57         ADD    r8, r5, r6, LSL #1
58         ADD    r8, r8, r8, LSL #1
59         ADD    r8, r8, r7
60
61         STRH   r8, [r3], #2        ; 1???? ??
62

```

```

63     LDRB     r5, [r0], #1        ; R
64     LDRB     r6, [r1], #1        ; G
65     LDRB     r7, [r2], #1        ; B
66
67     ; grayscale = (3R + 6G + B) >> 3
68     ADD      r8, r5, r6, LSL #1
69     ADD      r8, r8, r8, LSL #1
70     ADD      r8, r8, r7
71
72     STRH     r8, [r3], #2        ; 1???? ??
73
74
75
76     LDRB     r5, [r0], #1        ; R
77     LDRB     r6, [r1], #1        ; G
78     LDRB     r7, [r2], #1        ; B
79
80     ; grayscale = (3R + 6G + B) >> 3
81     ADD      r8, r5, r6, LSL #1
82     ADD      r8, r8, r8, LSL #1
83     ADD      r8, r8, r7
84
85     STRH     r8, [r3], #2
86
87     LDRB     r5, [r0], #1        ; R
88     LDRB     r6, [r1], #1        ; G
89     LDRB     r7, [r2], #1        ; B
90
91     ; grayscale = (3R + 6G + B) >> 3
92     ADD      r8, r5, r6, LSL #1
93     ADD      r8, r8, r8, LSL #1
94     ADD      r8, r8, r7
95
96     STRH     r8, [r3], #2
97
98     LDRB     r5, [r0], #1        ; R
99     LDRB     r6, [r1], #1        ; G
100    LDRB     r7, [r2], #1        ; B
101
102    ; grayscale = (3R + 6G + B) >> 3
103    ADD      r8, r5, r6, LSL #1
104    ADD      r8, r8, r8, LSL #1
105    ADD      r8, r8, r7
106
107    STRH     r8, [r3], #2
108
109    SUBS     r4, r4, #1
110    BNE      loop_gray
111
112    B loop_end
113
114    END

```

8. 프로젝트 수행일지

이름	유지원	김선영	박대호	최은지
모임일자	수행내용			
5/15	-프로젝트에 대해 논의 -각자 수행할 역할 분담 1): 최은지-Function1, 3에 대한 코드 작성, Function1 보고서 작성 2): 유지원-Function2에 대한 코드 작성, Function2 보고서 작성, ppt 제작 3): 김선영-Function3 보고서 작성, 발표, 수행일지 작성 4): 박대호-개요, 최적화 코드 작성, 보고서 취합			
5/29	-assembly 언어 27-bit RGB, 16-bit grayscale, pixel number 함수 작성 -assembly 언어 relocation 함수 작성 -relocation 과 3가지 function 프로젝트 구현			
5/31	-각자 작성한 보고서 공유, 피드백 후 취합			
6/4	-최종 보고서, ppt 제작			
6/7	-발표 대본 작성 -발표 영상 녹화			

9. Troubleshooting

- subproject 1 (작성자: 최은지)

처음에는 C코드와 어셈블리 코드를 연동하여 Red 값이 128 이상인 픽셀의 개수를 세고, 해당 결과를 UART로 출력하는 방식으로 구현을 진행하였다. 그러나 디버깅 과정에 4627이라는 값이 지속적으로 출력되는 문제가 발생하였다. 문제의 원인을 추적하던 중, Keil MDK의 기본 초기화 루틴인 startup.o 파일 내 __scatterload 함수가 .bss, .data 영역뿐만 아니라 .hex 파일을 통해 메모리에 로드한 RGBA 이미지 데이터까지 초기화하고 있다는 점을 확인하였다.

```
0x40000410: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x4000042A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x40000444: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x4000045E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x40000478: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x40000492: 36 F8 31 D9 10 1E A1 F7 5B DB F4 B1 CE 57 DB 47 19 D6 7C 13 1D CC 35 7D 6C AB
0x400004AC: DF C3 97 CA 2E 71 4D EC DA DB FE E3 48 B1 F6 3C 2B F6 24 7B 64 8E A5 C7 36 B5
0x400004C6: B1 6E 6F 3E 13 43 17 1C 23 16 6D 68 7B EF 1D 6E B9 BD C6 3C 71 DD 9D FC F3 B2

0x0000015C E24FE018 SUB R14,PC,#0x00000018
0x00000160 E3130001 TST R3,#0x00000001
0x00000164 1047F003 SUBNE PC,R7,R3
0x00000168 E12FFF13 BX R3
0x0000016C 000005B8 (???)EQ
0x00000170 000005C8 ANDEQ R0,R0,R8,ASR #11
        __scatterload_zeroinit:
0x00000174 E3B03000 MOVS R3,#0x00000000
0x00000178 E3B04000 MOVS R4,#0x00000000
0x0000017C E3B05000 MOVS R5,#0x00000000
0x00000180 E3B06000 MOVS R6,#0x00000000
0x00000184 E2522010 SUBS R2,R2,#0x00000010
0x00000188 28A10078 STMCSIA R1!,{R3-R6}
0x0000018C 8AFFFFFC BHI 0x00000184
0x00000190 E1B02E82 MOVS R2,R2,LSL #29
0x00000194 28A10030 STMCSIA R1!,{R4-R5}
0x00000198 45813000 STRMI R3,[R1]
```

```
AREA RESET, CODE, READONLY
ENTRY
EXPORT __main
IMPORT main

__main
    BL    main
    B     .
END
```

따라서 Keil의 기본 startup 코드를 제거하고, 직접 작성한 startup.s 파일을 통해 main 함수만 호출하는 최소한의 초기화 구조로 수정하였다. 하지만 이 과정에서 UART 초기화 함수인 init_serial()이 호출되지 않아 UART 출력이 전혀 동작하지 않는 문제가 발생했다.

```
main.s
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
238
```

```
UART #2
Number of pixels_red valued(relocation) >= 128: 4803
```

UART 출력값에 오류가 없었지만 팀원들과의 코드 교류이 일관성 확보 또 다른 문제의 발생 가능성 등을 고려해 C코드를 완전히 제거하고 어셈블리 코드만으로 동작하도록 구현 방식을 바꾸었다. 이 방식에서는 조건을 만족하는 픽셀 수를 계산한 후, 결과값이 저장된 레지스터(r0)의 16진수 값을 직접 확인하고, 이를 10진수로 변환하여 결과를 확인할 수 있다.

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000064
CPSR	0x60000003
SPSR	0x00000000

이를 통해 정확한 결과값인 4803을 확인할 수 있었으며, 문제의 근본 원인을 시스템 초기화 코드로부터 찾아내고 해결한 의미 있는 경험이 되었다.

- subproject 2 (작성자: 유지원)

error: L6235E: More than one section matches selector - cannot all be FIRST/LAST

원인 -> 초기 설정에서 `assem.s` 파일 이외에도 `startup.o` 또는 `serial.o`와 같은 소스 파일이 링크 대상에 포함되어 있었고, 이들 중 하나가 이미 `RESET`, `+First`와 같은 엔트리 포인트를 가지고 있었다. 이로 인해 링커가 진입점을 중복 인식하면서 충돌이 발생했다.

해결 방법: `assem.s` 내에서 `ENTRY`와 `EXPORT`만 명확히 지정하고, 프로젝트 구성에서 `startup.o`, `serial.o` 파일을 제거하여 중복된 엔트리 포인트 문제를 해결하였다.

- subproject 3 (작성자: 김선영)

32-bit RGBA image → 16-bit grayscale image의 이미지 변환 함수를 처음엔 C 코

드로 작성하였으나, 시뮬레이션이 잘 이루어지지 않아 어셈블리어로 그레이스케일 변환 함수를 다시 구현했다. 이후 ARM code 최적화를 위한 함수도 C 코드로 작성한 후 진행하려 하였지만 앞과 같은 문제점이 발생하여 어셈블리어로 함수를 작성했다. 함수를 C 코드에서 어셈블리어로 바꾸어 진행하여서 프로젝트의 결과를 잘 도출해낼 수 있었다.

-추가 최적화 (작성자: 박대호)

ARM모드 최적화 완료 이후 Thumb 모드를 기반으로 이미지를 처리하는 방식을 시도하였다. 그러나 Thumb-1 명령어 셋을 사용할 경우 R0부터 R7까지만 레지스터를 사용할 수 있는 구조적 제약이 있어 복잡한 계산이나 중간 값 보관에 어려움이 따랐다. 또한, Thumb 기반 코드에서 반복문 처리 중 프로그램이 중단되거나 디버그 세션이 비정상 종료되는 문제가 반복적으로 발생하여, 실습의 안정성과 확장성을 확보하기 어렵다고 판단하였다. 이에 따라 이후 ARM 모드 최적화로 만족하였다.

실습 과정 중 메모리 맵 설정에 있어서도 어려움이 있었다. 0x50000000 주소를 사용하기 이전에, 기존에 Keil이 내부적으로 사용하는 영역에 중복 등록을 시도함으로써 메모리 충돌 또는 접근 권한 문제가 발생하였다. 특히, 해당 주소 영역에 코드를 실행하거나 데이터를 쓰는 경우, 디버거가 내부적으로 사용 중인 공간이 함께 오염되어 메모리 내용이 예기치 않게 변경되는 문제가 발생하였다. 이와 같은 문제를 해결하기 위해 메모리 매핑은 기존 시스템이 사용하지 않는 빈 주소 영역(0x50000000, 0x20000000 등)으로 명확히 분리하여 등록하였고, READWRITE 권한을 명시적으로 부여함으로써 안정적인 디버깅 환경을 구축할 수 있었다.